

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Магнитогорский государственный технический университет им. Г.И. Носова»
(ФГБОУ ВО «МГТУ им. Г.И. Носова»)

Институт энергетики и
автоматизированных систем
Кафедра вычислительной техники и
программирования
Направление 09.03.01 – Информатика
и вычислительная техника

Допустить к защите
Заведующий кафедрой
_____ /О. С. Логунова/
«_____» _____ 2026 г.

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

обучающегося Афонькина Максима Артемовича

на тему: «Проектирование и разработка WEB–приложения для рецензирования музыкального творчества»

ВКР выполнена на ___ страницах

Графическая часть на ___ листах

Руководитель: доцент кафедры ВТ и П, к.т.н. Егорова Л. Г

(подпись, дата, должность, ученая степень, ученое звание, Ф.И.О)

Нормоконтроль и проверка
на антиплагиат выполнены.

Оригинальность текста ___ %

_____/Л. Г. Егорова/

(подпись, дата)

Обучающийся _____

(подпись)

«_____» _____ 2026 г.

Министерство науки и высшего образования Российской Федерации

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«Магнитогорский государственный технический университет им. Г.И. Носова»
(ФГБОУ ВО «МГТУ им. Г.И. Носова»)

Кафедра вычислительной техники и
программирования

УТВЕРЖДАЮ:

Заведующий кафедрой

_____/О. С. Логунова/

« ____ » _____ 2026 г.

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

ЗАДАНИЕ

Тема: «Проектирование и разработка WEB–приложения для рецензирования музыкального творчества»

Обучающемуся Афонькин Максиму Артемовичу

Тема утверждена приказом № 10–35/242 от 27.02.2026 г.

Срок выполнения «__» _____ 2026 г.

Исходные данные к работе:

1. ГОСТ 19.701–90 «Схемы алгоритмов, программ, данных и систем. Обозначения условные и правила выполнения».

2. СМК–О–СМГТУ–36–20 «Выпускная квалификационная работа: структура, содержание, общие правила выполнения и оформления».

Перечень вопросов, подлежащих разработке в выпускной квалификационной работе:

1. Теоретический анализ существующих бизнес–процессов и проблемных зон в работе приёмной комиссии на примере МГТУ им. Г.И. Носова.

2. Формирование технических и функциональных требований к разрабатываемой системе, разработка web–приложения и представление порядка работы с ним.

3. Тестирование и опытная эксплуатация разработанного web–приложения.

Графическая часть:

Руководитель: _____ / Л. Г. Егорова /
(подпись, дата)

Задание получил: _____ / М.А. Афонькин /
(подпись, дата)

Министерство науки и высшего образования Российской Федерации

Федеральное государственное бюджетное образовательное
учреждение высшего образования
«Магнитогорский государственный технический университет им. Г.И. Носова»
(ФГБОУ ВО «МГТУ им. Г.И. Носова»)

Институт энергетики и автоматизированных систем
Кафедра вычислительной техники и программирования

Календарный график

выполнения выпускной квалификационной работы

Обучающегося Афонькина Максима Артемовича, студента 4 курса группы АВб–22–2
(фамилия, имя, отчество, курс, факультет, группа)

Тема ВКР: Проектирование и разработка WEB–приложения для рецензирования музыкаль-
ного творчества

(полное наименование темы выпускной квалификационной работы в соответствии с приказом об утвер-
ждении тем ВКР и назначении руководителей)

п/п	Этапы выполнения ВКР	Дата (срок) выполнения		Отметка руководителя ВКР или заведующего кафедрой о выполнении
		план	факт	
1.	Разработка структуры ВКР. Проведение анализа уровня разработанности проблем	03.03.2026	03.03.2026	
2.	Сбор фактического материала (лабораторные, исследовательские работы и др.)	20.03.2026	20.03.2026	
3.	Подготовка рукописи ВКР	05.05.2026	05.05.2026	
4.	Доработка текста ВКР в соответствии с замечаниями руководителя	25.05.2026	25.05.2026	
5.	Ознакомление с отзывом руководителя и рецензией	05.06.2026	05.06.2026	
6.	Подготовка доклада и презентационного материала	15.06.2026	15.06.2026	

Обучающийся _____ М.А. Афонькин
(подпись)

Руководитель ВКР _____ Л.Г. Егорова
(подпись)

Заведующий кафедрой _____ О.С. Логунова
(подпись)

ОТЗЫВ

на выпускную квалификационную работу студента гр. АВб–22–2

Института энергетики и автоматизированных систем

ФГБОУ ВО «МГТУ им. Г.И. Носова»

Афонькина Максима Артемовича

на тему «Проектирование и разработка WEB–приложения для рецензирования музыкального творчества»

Целью выпускной квалификационной работы является повышение объективности, информативности и доступности процессов рецензирования русскоязычных музыкальных произведений молодёжной аудиторией за счёт разработки специализированного web–приложения с многокритериальной оценкой, модерацией, музыкальным каталогом и пользовательскими профилями.

Работа выполнена качественно, в соответствии с заданием, отличается логичностью, полнотой изложения и глубокой проработкой материала. К сильным сторонам работы можно отнести комплексную реализацию проекта: в системе объединены пользовательские сценарии рецензирования, рейтинговая модель, административные функции, элементы геймификации и средства контейнерного развертывания.

В первой главе рассмотрены предметная область, особенности онлайн–рецензирования музыкального творчества, существующие web–платформы и обоснована необходимость разработки специализированного решения для русскоязычной аудитории.

Во второй главе описаны проектирование и разработка приложения: технологический стек, архитектура, структура базы данных, пользовательские сценарии, модель оценки, геймификация, административные функции и средства развертывания.

В третьей главе представлены результаты опытной эксплуатации web–приложения, демонстрационные данные, основные пользовательские и административные сценарии, а также результаты проверки интерфейса, сборки и развертывания. В качестве дальнейшего развития можно отметить расширение аналитики и пользовательской статистики.

Считаю, что выпускная квалификационная работа, выполненная Афонькиным М.А., заслуживает оценки «отлично», а выпускник – присвоения квалификации «бакалавр» по направлению «Информатика и вычислительная техника».

Руководитель выпускной квалификационной работы:

Доцент кафедры ВТиП, к.т.н. _____ /Л.Г. Егорова/

РЕФЕРАТ

Тема выпускной квалификационной работы: «Проектирование и разработка WEB–приложения для рецензирования музыкального творчества».

Пояснительная записка содержит: 87 страниц, 28 иллюстраций, 23 таблицы, 3 приложения, 30 использованных источников.

Ключевые слова: музыкальное творчество, рецензирование, web–приложение, рейтинговая оценка, музыкальный каталог, модерация, пользовательский профиль, геймификация, Docker, CI/CD.

Актуальность выбранной темы заключается в том, что в современной цифровой среде музыкальные произведения активно обсуждаются пользователями, однако такие обсуждения часто остаются разрозненными и слабо структурированными.

Объектом исследования является процесс рецензирования музыкального творчества на web–платформах.

Предметом исследования является разрабатываемое web–приложение, предназначенное для публикации рецензий, выставления многокритериальных оценок, модерации контента, работы с музыкальным каталогом и формирования аналитических показателей.

Целью выпускной квалификационной работы является повышение объективности, информативности и доступности рецензирования русскоязычных музыкальных произведений за счёт разработки специализированной информационной системы многокритериального оценивания.

Для достижения поставленной цели были определены следующие задачи:

1. Теоретический анализ предметной области и существующих решений для рецензирования музыкального творчества.
2. Формирование технических и функциональных требований к разрабатываемой системе, разработка web–приложения и представление порядка работы с ним.
3. Тестирование и опытная эксплуатация разработанного web–приложения.

СОДЕРЖАНИЕ

Введение.....	9
1 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ И АНАЛИЗ СУЩЕСТВУЮЩИХ РЕШЕНИЙ РЕЦЕНЗИРОВАНИЯ МУЗЫКАЛЬНОГО ТВОРЧЕСТВА	11
1.1 Описание предметной области	11
1.2 Анализ исходных данных и информационных потоков в процессе рецензирования музыкального творчества на веб–платформах	15
1.3 Оценка существующих веб–платформ и практических решений для рецензирования музыкальных произведений	21
1.4 Выводы по первой главе	27
2 ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА ИНФОРМАЦИОННОЙ СИСТЕМЫ РЕЦЕНЗИРОВАНИЯ МУЗЫКАЛЬНОГО ТВОРЧЕСТВА	30
2.1 Архитектура веб–приложения	30
2.2 Функциональные требования и сценарии использования	31
2.3 Проектирование базы данных	34
2.4 Модель рейтинговой оценки и геймификации	39
2.4.1 Модель рейтинговой оценки.....	39
2.4.2 Геймификация и профиль пользователя.....	42
2.5 Разработка серверной части веб–приложения	44
2.6 Разработка клиентской части веб–приложения	49
2.7 Разработка административной панели.....	51
2.8 DevOps–инструменты и воспроизводимость развертывания.....	52
2.9 Выводы по второй главе.....	55
3 РЕЗУЛЬТАТЫ ОПЫТНОЙ ЭКСПЛУАТАЦИИ И ПРОВЕРКА РАБОТОСПОСОБНОСТИ СИСТЕМЫ.....	58
3.1 Подготовка демонстрационных данных.....	58

3.2 Пользовательский сценарий работы с системой	59
3.3 Сценарий работы администратора	63
3.4 Проверка качества интерфейса.....	67
3.5 Проверка сборки и развертывания	68
3.6 Анализ результатов и ограничения	71
3.7 Выводы по третьей главе	72
Заключение	74
Список используемых источников.....	76
Приложение А «Листинг конфигурации CI/CD пайплайна проекта».....	82
Приложение Б «Листинг кода алгоритма расчета рейтинговой оценки» ..	84
Приложение В «Листинг кода Расчета опыта и статистики профиля пользователя»	85

ВВЕДЕНИЕ

С развитием информационных технологий автоматизация процессов обработки пользовательского контента становится ключевым фактором повышения эффективности веб-платформ. Ручная работа с рецензиями, рейтингами и модерацией постепенно уступает место специализированным системам, особенно в нишевых сообществах.

Русскоязычная музыкальная сфера, испытывает дефицит качественных платформ для коллективного рецензирования. Молодёжная аудитория (18–25 лет) активно потребляет современную русскую музыку, но существующие ресурсы либо зарубежные (RateYourMusic – 1,3 млн пользователей), либо универсальные стриминги без развитой системы пользовательских отзывов. По данным IFPI 2025, глобальные доходы от музыки достигли \$29,6 млрд, из которых стриминг – 69%, но русскоязычные платформы рецензий практически отсутствуют.

Актуальность темы обусловлена потребностью русскоязычной молодёжи в специализированном портале для экспертного анализа современной музыки. За 2015–2025 годы аудитория стриминговых сервисов выросла в 10 раз (до 752 млн подписчиков), но ручная модерация рецензий приводит к системным проблемам: публикация низкокачественного контента; отсутствие унифицированной системы экспертной оценки; задержки в утверждении материалов; недостаточная фильтрация по музыкальным направлениям.

Отсутствие полноценной русскоязычной платформы с системной модерацией и экспертной системой оценки создаёт объективную потребность в самостоятельном веб-продукте, ориентированном на качественное рецензирование музыкального творчества.

Объектом исследования данной выпускной квалификационной работы является процесс коллективного рецензирования музыкального творчества русскоязычной молодёжью на веб-платформах.

Предметом исследования является информационная система рецензирования современной музыки с экспертной системой оценки и инструментами модерации пользовательского контента.

Целью выпускной квалификационной работы является повышение объективности, информативности и доступности процессов рецензирования русскоязычных музыкальных произведений молодёжной аудиторией за счёт разработки и внедрения специализированной информационной системы многокритериального оценивания, обеспечивающей сокращение времени получения и обработки рецензий, повышение достоверности и защищённости оценочных данных, а также расширение набора аналитических показателей для последующего анализа музыкального контента.

Для достижения поставленной цели решаются следующие задачи:

1. Анализ ниши русскоязычного рецензирования современной музыки и проблем существующих платформ.

2. Проектирование архитектуры системы с экспертной оценкой (рифмы/образы, структура/ритмика, реализация стиля, индивидуальность/харизма, атмосфера/вайб), инструментами модерации и фильтрами жанров.

3. Реализация прототипа, интеграция каталогов музыкальных произведений, тестирование модерации и пользовательского интерфейса.

Разработанный портал заполнит нишу качественного рецензирования русской музыки для молодёжи, обеспечив экспертную систему оценки и инструменты модерации контента, соответствующие профилю «Проектирование и разработка Web–приложений».

1 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ И АНАЛИЗ СУЩЕСТВУЮЩИХ РЕШЕНИЙ РЕЦЕНЗИРОВАНИЯ МУЗЫКАЛЬНОГО ТВОРЧЕСТВА

1.1 Описание предметной области

Современная музыкальная блогосфера и рынок стриминговых сервисов представляют собой динамично развивающуюся экосистему веб–платформ и социальных сервисов, обеспечивающих взаимодействие между слушателями, авторами контента и музыкальными критиками. В России этот сегмент переживает качественные трансформации: по данным TopHit Spy, в 2024 году русскоязычные композиции заняли 75% позиций в радиоэфире Топ–20 городов, обогнав зарубежных исполнителей в 3,3 раза. Объём рынка музыкального стриминга достиг 38 млрд рублей (+47,8% к 2023 году), демонстрируя устойчивый рост с 2015 года, когда показатель составлял менее 5 млрд рублей.

Молодёжная аудитория в возрасте 18–25 лет проводит с музыкой в среднем около четырёх часов ежедневно. При этом 97% респондентов отмечают, что обращаются к ней постоянно в течение дня или, как минимум, несколько раз в сутки. Эта группа формирует музыкальные тренды через социальные сети и стриминговые платформы, где хип–хоп и рэп стабильно занимают 16–18 позиций из 30 в топ–чартах. За десятилетие (2015–2025) доля платных подписок выросла с 5% до 47%, а ежедневное потребление увеличилось в 2,5 раза, что свидетельствует о зрелости рынка и потребности в специализированных инструментах анализа контента [1].

Русскоязычная музыкальная блогосфера остаётся фрагментированной: контент рассеян между стриминговыми сервисами, социальными сетями и разрозненными ресурсами. Крупные платформы предлагают базовые пользовательские отзывы, но лишены централизованной системы экспертного рецензирования. Одним из немногих известных ресурсов подобного типа является RateYourMusic, ориентированный на англоязычную аудиторию, тогда как отечественные сервисы фокусируются на прослушивании без развитых инструментов

глубокого анализа. В результате профессиональная музыкальная критика в русскоязычном сегменте оказывается распределённой по разным площадкам и не опирается на общепринятую инфраструктуру.

Бизнес–процессы рецензирования в музыкальной блогосфере включают полный жизненный цикл пользовательского контента: отбор материала из каталогов с фильтрацией по жанрам, экспертный анализ по специализированным критериям, контроль качества с исключением низкокачественных материалов, публикацию утверждённых рецензий с системой лайков и комментариев, а также формирование персонализированных рекомендаций на основе популярности. В качестве критериев оценки могут использоваться рифмы и образы, структура и ритмика, реализация стиля, индивидуальность исполнителя, общая атмосфера произведения, что позволяет формализовать субъективные впечатления и перенести их в единую систему оценок. Традиционные платформы при этом предлагают упрощённую звёздочную оценку или несогласованные текстовые отзывы без унифицированных методик, что снижает объективность и доверие аудитории.

Зарубежные платформы рецензирования, такие как RateYourMusic и AllMusic, аккумулируют значительный объём пользовательских и экспертных оценок и фактически выступают ориентиром для англоязычной аудитории, однако они слабо учитывают специфику русскоязычного музыкального пространства и ориентированы преимущественно на западный каталог релизов. На российском рынке доминируют VK Музыка и Яндекс Музыка, которые обладают крупной аудиторией и развитыми рекомендациями, но в них отсутствует полноценная система формализованных рецензий: пользователь ограничен лайком, добавлением в плейлист и кратким комментарием, что не позволяет вести структурированное обсуждение качества музыкальных произведений и альбомов [2].

В условиях такой фрагментации молодёжная аудитория вырабатывает собственные стратегии выбора музыки: ключевую роль играют рекомендательные ленты стримингов, тематические плейлисты настроений, короткие видео в соци-

альных сетях и мнения блогеров. Значительная часть новых композиций попадает к слушателю через алгоритмические подборки вроде персонализированных миксов и разделов «новинки недели». Важную роль играют и короткие вирусные видеоролики на развлекательных платформах (включая сервисы формата TikTok), где композиция закрепляется в сознании прежде всего как «саундтрек к тренду», а не как цельное художественное произведение. В результате многие пользователи делают выбор на основе обложки, фрагмента припева или рекомендаций алгоритма, не имея под рукой развёрнутых рецензий, которые помогли бы оценить текст, звук и авторский замысел до того, как слушатель потратит время на полное знакомство с произведением.

Для рядового слушателя это означает не столько свободу выбора, сколько постоянное состояние информационной перегрузки: каталоги насчитывают миллионы музыкальных произведений, а алгоритмы рекомендаций подбрасывают всё новые композиции быстрее, чем пользователь успевает их осмысленно оценить. В такой среде решение «слушать или пролистать дальше» чаще принимается за считанные секунды по косвенным признакам – обложке, названию, жанровой метке, кратким комментариям других пользователей, тогда как целостная картина сильных и слабых сторон произведения остаётся за кадром. Это формирует устойчивый запрос на структуры, которые не заменяют стриминги, а дополняют их слоем систематизированных рецензий и позволяют быстро понять, чем именно интересна конкретная композиция или альбом.

Подписочная модель особенно остро проявляет разрыв между техническим доступом к музыке и качеством среды вокруг неё. Исследования показывают, что значительная доля россиян уже оформляет платные подписки на музыкальные сервисы, причём молодёжь чаще других использует их ежедневно. Однако внутри этих платформ пользователь по-прежнему остаётся в роли пассивного потребителя: он может поставить отметку «нравится» или пропустить произведение. При этом он не получает полноценных инструментов, чтобы развернуто высказать своё мнение, сравнить его с оценками ровесников и вести содержательную дискуссию.

В результате формируется неостребованный потенциал активной аудитории, которой нужна не только «музыкальная лента», но и специализированное пространство для аргументированного обмена рецензиями, где каждый слушатель может быть услышан и одновременно опираться на мнения других.

В этих условиях перспективной становится модель рецензирования, опирающаяся не на формальную пятибалльную шкалу, а на набор критериев, ближе к тому, как молодёжь реально воспринимает музыку. Оценка текста, образной системы и рифмовки, отдельная фиксация структуры и ритмики, реализации жанрового стиля, индивидуальности исполнителя и общей атмосферы произведения позволяет разложить впечатление на понятные составляющие и показать, за что именно композиция ценится слушателями. Такая многокритериальная система не выглядит академической или оторванной от жизни: она говорит на привычном языке «вайба», подачи и самобытности, создаёт основу для осмысленного сравнения разных работ внутри одного жанра и открывает возможность живого диалога между исполнителями и аудиторией на базе прозрачных, разделяемых сообществом ориентиров качества.

Важно, что такая система рецензирования открывает возможности не только для слушателей, но и для самих исполнителей. В отличие от разрозненных комментариев в социальных сетях, структурированные отзывы с разбиением по критериям позволяют артисту видеть, какие именно аспекты его творчества находят отклик у аудитории: текст, образность, подача, звук или общая атмосфера произведения. Наличие персонализированных профилей исполнителей и их взаимодействие с разделом рецензий создаёт формат содержательного диалога, когда музыкант может отвечать на аргументированные отклики, уточнять замысел, а активные слушатели – чувствовать себя не безликими слушателями потока, а участниками сообщества, влияющего на развитие современного музыкального контента.

Таким образом, предметная область рецензирования музыкального творчества в русскоязычной цифровой среде сочетает высокую степень развития стриминговых сервисов и вовлечённости молодёжной аудитории с отсутствием

специализированной инфраструктуры для системного анализа и обсуждения произведений. Наличие фрагментированных отзывов, упрощённых схем оценивания и пассивной роли пользователя формирует устойчивый запрос на информационную систему, которая объединит многокритериальную оценку, механизмы модерации и каналы содержательного общения исполнителей со слушателями в едином веб–пространстве, ориентированном на современное музыкальное сообщество.

1.2 Анализ исходных данных и информационных потоков в процессе рецензирования музыкального творчества на веб–платформах

Под организацией процесса рецензирования музыкального творчества на веб–платформах в данной работе понимается совокупность регламентов, пользовательских сценариев и технических средств. Они определяют путь произведения от момента его выбора слушателем до появления связанной с ним рецензии и последующей реакции аудитории. В этот процесс входят поиск и идентификация музыкального произведения в каталоге, привязка отзыва к конкретному альбому или композиции, оформление текстового комментария и оценок, проверка материалов модерацией (если она предусмотрена), публикация рецензии и дальнейшее взаимодействие других пользователей с этим контентом – от простых отметок «нравится» до развёрнутых дискуссий в комментариях. Таким образом, организация процесса рецензирования напрямую влияет на удобство работы пользователей с платформой и на качество формируемой системы оценок и отзывов о музыкальных произведениях [3].

На большинстве существующих ресурсов, ориентированных на оценку музыки, путь пользователя строится по схожей схеме. Слушатель выбирает в поиске исполнителя и нужный альбом или композицию, переходит на страницу произведения, где видит обобщённую информацию (название, год выпуска, список композиций, жанровые метки), среднюю пользовательскую оценку и уже оставленные рецензии. Как правило, на этой странице располагается форма для

быстрого выставления общей числовой оценки и поле для текстового комментария. Пример интерфейса страницы музыкального альбома с агрегированной оценкой и пользовательскими рецензиями представлен на рисунке 1.

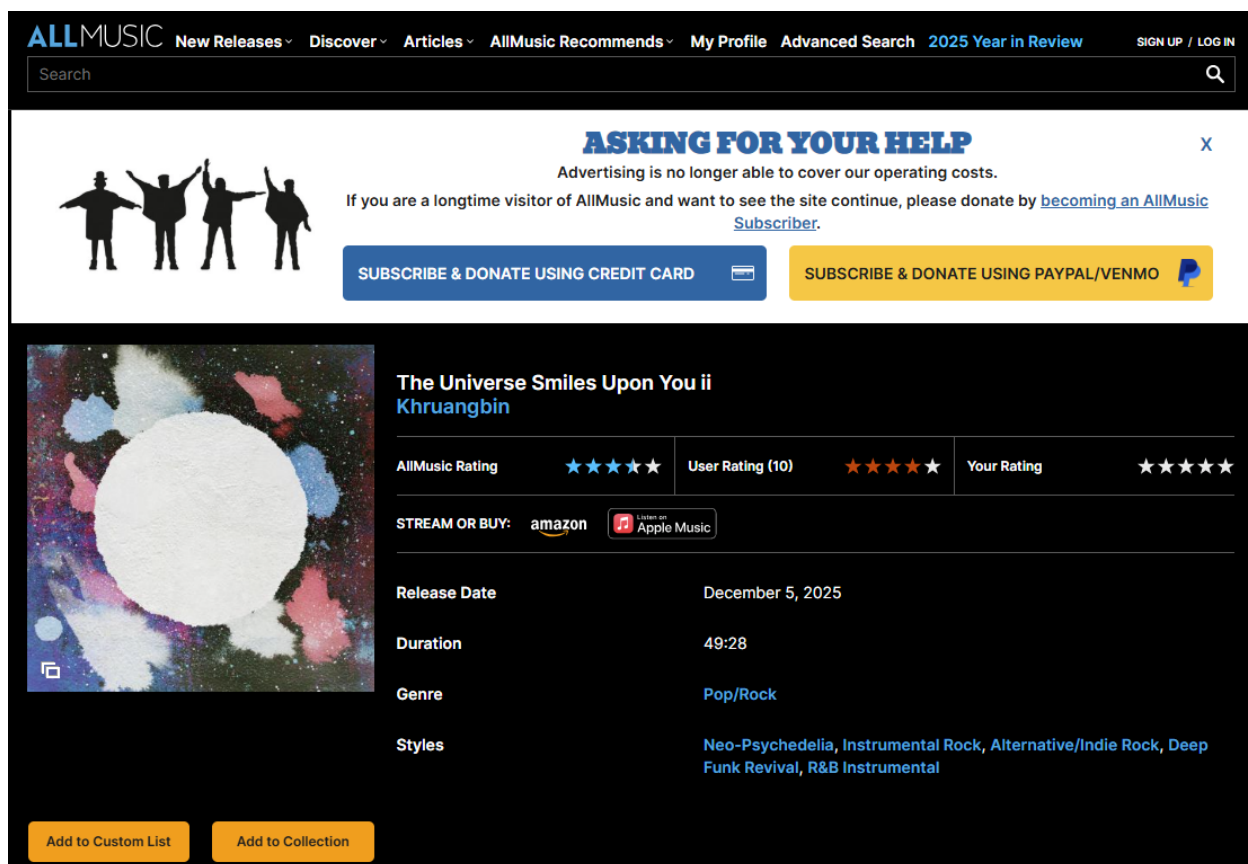


Рисунок 1 – Страница альбома на веб-платформе AllMusic

Несмотря на визуальную наглядность таких страниц, внутренняя структура пользовательской оценки на большинстве платформ остаётся однородной и малоинформативной. Слушателю предлагается зафиксировать своё отношение к альбому или композиции в виде одной интегральной отметки – набора звёзд или значения на числовой шкале, в то время как отдельные аспекты произведения (текст, вокал, аранжировка, общая атмосфера) в явном виде нигде не выделяются. Текстовые рецензии, как правило, не разделены на смысловые блоки и заполняются в одном свободном поле, из-за чего разные авторы используют произвольную форму изложения: от кратких реплик «понравилось / не понравилось» до объёмных обзоров, построенных по собственным неформальным правилам.

На практике это приводит к преобладанию коротких, слабо структурированных отзывов, где общая звёздная оценка дополняется несколькими предложениями свободного комментария без явного указания, какие именно стороны произведения оцениваются. Пользователь, как правило, ограничивается общими формулировками вроде «приятно слушать» или «не зацепило», не разделяя впечатления по тексту, звучанию или подаче, а другие слушатели могут лишь отметить полезность отзыва с помощью кнопок «Yes/No», не влияя на его доработку или уточнение. Пример такой пользовательской рецензии на музыкальный альбом с общей звёздной оценкой и кратким текстовым комментарием без деления на критерии представлен на рисунке 2.

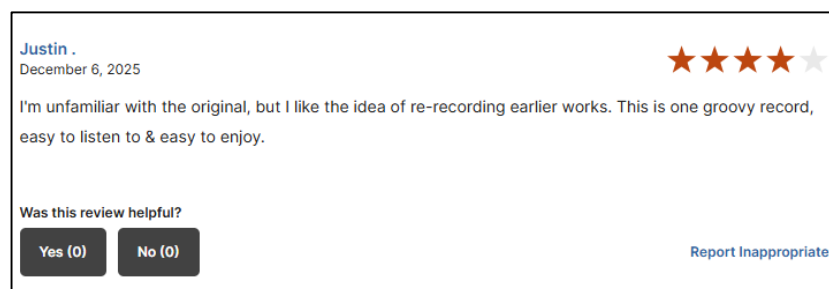


Рисунок 2 – Пример пользовательской рецензии на музыкальный альбом на платформе AllMusic

На большинстве существующих платформ все участники сообщества формально обладают одинаковыми правами на публикацию рецензий и выставление оценок, однако при этом почти отсутствуют механизмы управляемого влияния на качество контента. Независимо от опыта и стиля письма пользователя его отзыв попадает в общий поток, а модерация в основном ограничивается реакцией на очевидные нарушения правил (спам, оскорбления, реклама), не затрагивая вопросы информативности, обоснованности и оригинальности текста. В результате слабые или однотипные комментарии легко «размывают» интересные, нестандартные рецензии, а у исполнителей практически нет инструментов, позволяющих выделить действительно содержательные отклики – например, отметить понравившуюся рецензию так, чтобы она была заметна остальным слушателям.

Важным следствием такой организации является то, что у пользователей практически нет возможностей выделиться за счёт качественной, вдумчивой рецензии. Даже хорошо написанный обзор, в котором автор честно указывает на сильные и слабые стороны произведения, в интерфейсе большинства платформ ничем не отличается от десятков коротких комментариев, а позиция исполнителя по отношению к этим отзывам остаётся неявной. Музыкант, даже если он ценит аргументированную обратную связь, не располагает явным инструментом, позволяющим отметить конкретный отзыв как особенно полезный для себя и тем самым показать остальному сообществу, что такая форма рецензирования поощряется [4].

На интерфейсном уровне это выражается в том, что рецензии отображаются в виде единообразного вертикального списка, где блоки с поверхностными комментариями и большие аналитические обзоры оформлены одинаково и занимают одно и то же место в общей ленте. Пользователь видит последовательность однотипных карточек с именем автора, звёздной оценкой и текстом, а возможность визуально отделить наиболее содержательные отклики от фона сведена к минимуму: максимум, что предлагается, это кнопки оценки полезности «Yes/No», не меняющие структуру списка.

Дополнительно следует отметить, что подобная организация снижает мотивацию пользователей к написанию развёрнутых отзывов. Если качественная рецензия визуально не отличается от короткого комментария и не получает отдельного статуса в интерфейсе, автору сложнее увидеть практическую ценность своей работы. В результате платформа постепенно смещается в сторону быстрых реакций, тогда как аналитическая составляющая рецензирования остаётся недостаточно выраженной.

Пример такого представления пользовательских рецензий в виде однородного перечня показан на рисунке 3.

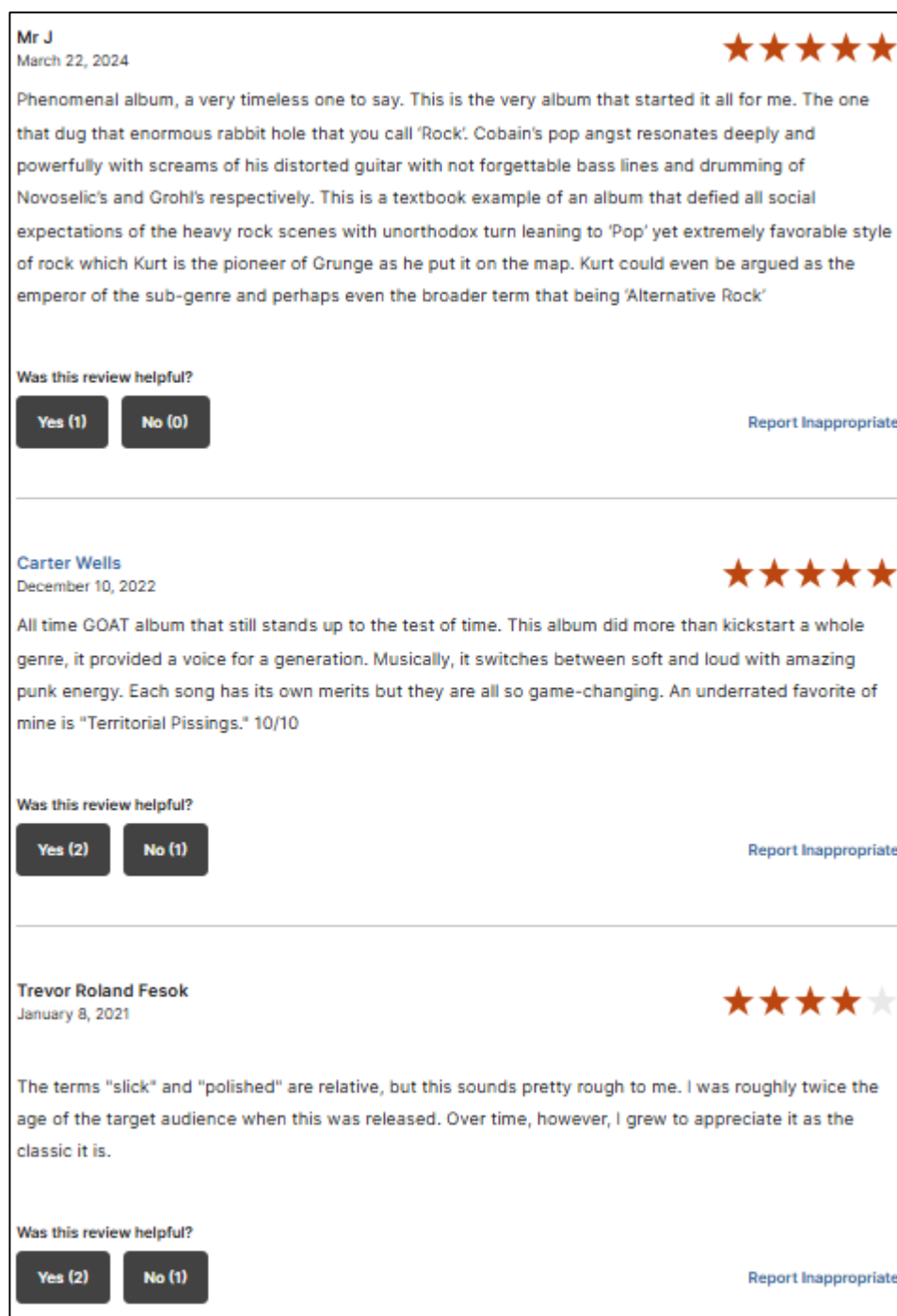


Рисунок 3 – Список пользовательских рецензий на музыкальный альбом на платформе AllMusic

Отдельного внимания заслуживает визуальная организация страниц, на которых размещаются рецензии. На ряде популярных ресурсов значительную часть экрана занимают служебные элементы и рекламные блоки: массивный баннер в верхней части страницы, дополнительные панели навигации, промо–

вставки между разделами основного контента. При таком подходе блок с пользовательскими отзывами оказывается смещён вниз и визуально конкурирует с рекламой за внимание пользователя, что снижает комфорт восприятия и делает работу с рецензиями менее удобной для молодой аудитории, привыкшей к более лаконичным и фокусным интерфейсам. Характерный пример перегруженного интерфейса страницы альбома с рекламным баннером и смещённым вниз блоком рецензий показан на рисунке 4.

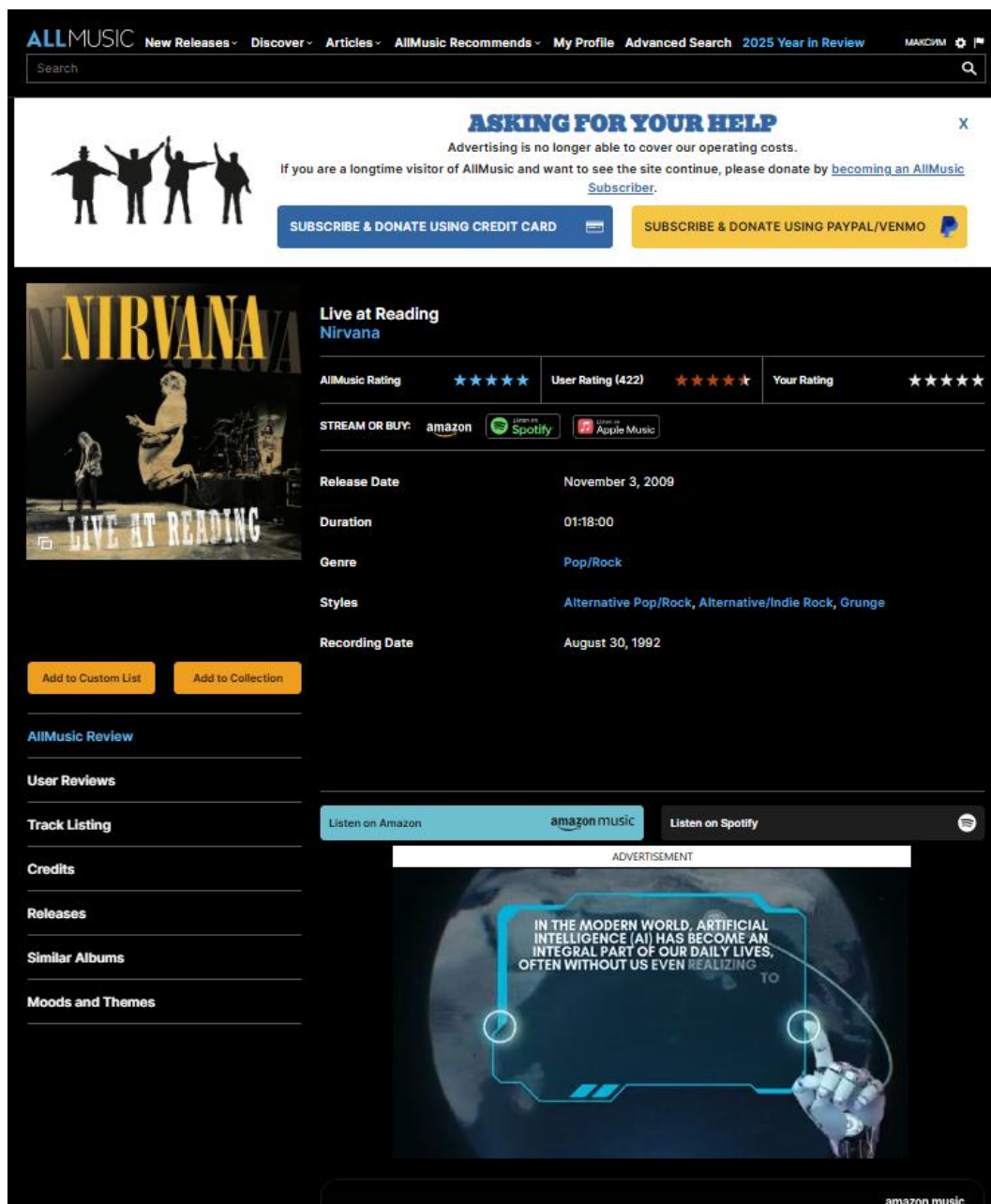


Рисунок 4 – Интерфейс страницы на AllMusic

В совокупности перечисленные особенности показывают, что существующие веб–платформы для рецензирования музыки ориентированы прежде всего на фиксацию факта оценки, а не на формирование удобной, структурированной среды для анализа и обсуждения произведений. Отсутствие явного многокритериального подхода, слабая поддержка выделения качественных рецензий и перегруженные интерфейсы затрудняют как осознанный выбор музыки со стороны слушателей, так и системное использование обратной связи со стороны исполнителей. Эти ограничения обуславливают необходимость разработки специализированной информационной системы рецензирования музыкального творчества, в которой логика работы с отзывами, критериями оценки и взаимодействием участников будет заложена изначально и рассмотрена в подразделе 1.3.

1.3 Оценка существующих веб–платформ и практических решений для рецензирования музыкальных произведений

Анализ предметной области и существующей организации процесса рецензирования показывает, что имеющиеся веб–платформы не удовлетворяют потребности русскоязычной молодёжной аудитории в осмысленном обсуждении современной музыки. Массовые стриминговые сервисы ориентированы преимущественно на потребление контента и простые количественные сигналы (прослушивания, лайки), тогда как специализированные сайты с пользовательскими рейтингами используют устаревшие интерфейсы и одноканальные шкалы оценивания, не отражающие реального богатства восприятия музыкального произведения. В результате рецензии остаются разрозненными и слабо структурированными, а их потенциал как инструмента культурного диалога и развития музыкального вкуса используется лишь частично [5].

Особенно остро эти проблемы проявляются в контексте современной русскоязычной сцены, где основными носителями музыкальных трендов выступают молодые слушатели в возрасте 18–25 лет. Для них музыка является не только фоном, но и способом самовыражения: важно не просто «поставить оценку», а сформулировать собственную позицию, получить содержательную обратную

связь от сверстников и, по возможности, быть замеченным самими исполнителями. Однако текущие платформы практически не предоставляют инструментов, которые поощряли бы написание продуманных рецензий и позволяли бы выделить качественные отзывы среди массового потока однотипных комментариев.

Для решения обозначенных проблем требуется специализированное веб-приложение, исходно ориентированное не на потоковое прослушивание, а на структурированное рецензирование музыкального материала. Такое приложение должно объединять в себе функции каталога произведений и исполнителей, среды для многокритериальной оценки и площадки для содержательного общения пользователей, при этом оставаясь достаточно простым и интуитивным, чтобы им могла пользоваться широкая молодёжная аудитория без специальной подготовки. В отличие от универсальных стриминговых сервисов, акцент в разрабатываемой системе переносится с пассивного потребления на осмысленное взаимодействие с музыкой и обмен аргументированными мнениями о ней [6].

Ключевым отличительным признаком такой информационной системы должна стать многокритериальная модель оценивания, отражающая реальные способы восприятия музыки молодёжной аудиторией. Вместо одной усреднённой звёздной отметки пользователю предлагается оценивать произведение по ряду аспектов: рифмы и образы, структура и ритмика, реализация стиля, индивидуальность и харизма исполнителя, общая атмосфера и эмоциональный «вайб» композиции. Такой подход отделяет эмоциональное впечатление от конкретных характеристик произведения и делает итоговую оценку более прозрачной для пользователя.

Интерфейс формы добавления рецензии в разрабатываемой системе реализует эти критерии в виде отдельных шкал с последующим расчётом итогового балла, что позволяет наглядно зафиксировать сильные и слабые стороны произведения и делает оценку более информативной для других пользователей и самих музыкантов; пример такой формы представлен на рисунке 5.

РЕЦЕНЗИЯ

Добавить рецензию

Итог 79

КРИТЕРИИ
Основная оценка

Сумма 40

Рифмы / образы 10

Структура / ритмика 10

Реализация стиля 10

Индивидуальность / харизма 10

Итоговый балл 79

PO	ST	PC	IX	AT
10	10	10	10	7

Итог складывается из базовых критериев и общего вайба. Детали видны в карточке после публикации.

ОБЩЕЕ ВПЕЧАТЛЕНИЕ
Атмосфера / вайб

Вайб 7

Атмосфера / вайб 7

☐ Добавить текстовую рецензию
Текстовая рецензия отправляется на модерацию.

Отмена Отправить

Рисунок 5 – Форма многокритериальной оценки музыкального произведения

Вторая важная задача разрабатываемого web–приложения связана с управлением качеством пользовательских рецензий. В отличие от существующих площадок, где модерация в основном ограничивается борьбой со спамом и нарушениями правил общения, в предлагаемой системе все новые текстовые рецензии сначала попадают в очередь на модерацию и не отображаются в общем списке до принятия решения. Модератор видит такие материалы в виде упорядоченного перечня с указанием автора, выбранного альбома, выставленных по критериям оценок и итогового балла и для каждой записи может явно отметить, допускается ли она к публикации или должна быть отклонена как недостаточно информативная или дублирующая уже существующие отзывы. Это позволяет поддерживать единый уровень качества опубликованных материалов и не перегружать публичную ленту случайными или неполными отзывами. Интерфейс панели модератора, отображающий список рецензий «на модерации» и доступные действия по их утверждению или отклонению, представлен на рисунке 6.

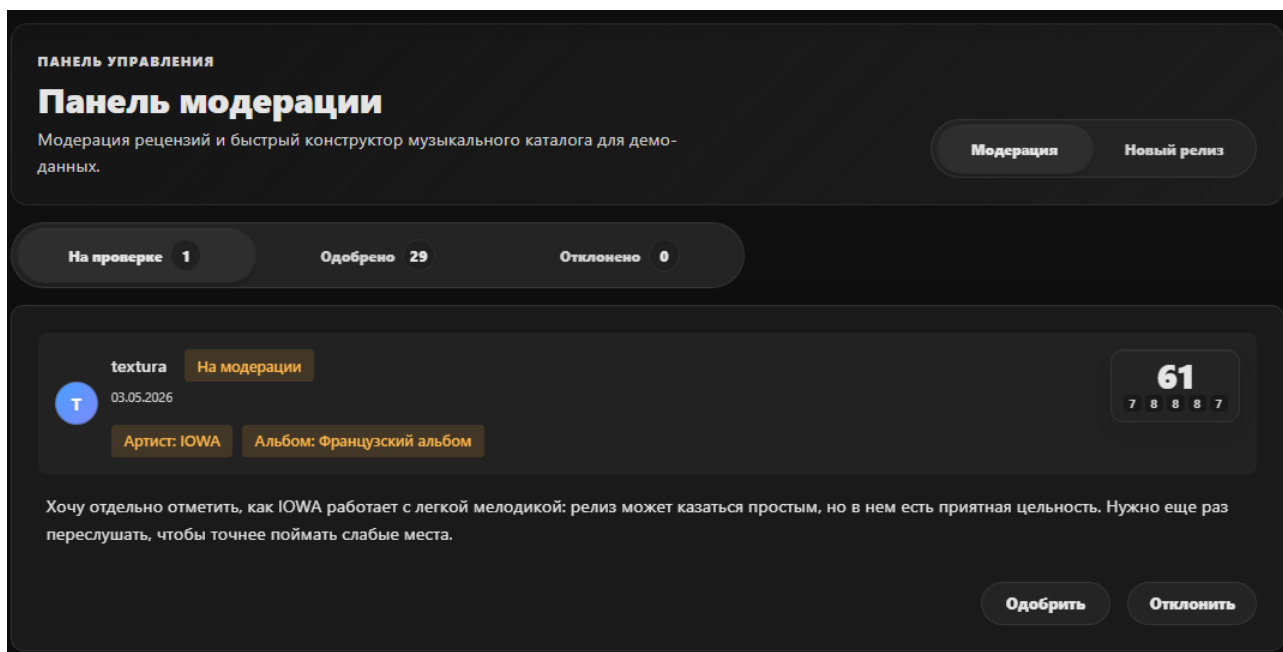


Рисунок 6 – Рецензии ожидающие проверки

Отдельное внимание в проектируемой системе уделяется выстраиванию осмысленного диалога между слушателями и исполнителями. В дополнение к обычным пользовательским откликам вводится специальная отметка со стороны автора произведения, с помощью которой музыкант может выделить те рецензии, которые оказались для него особенно полезными или точно сформулировали обратную связь. В отличие от обычного лайка, такая реакция имеет больший смысловой вес, поскольку показывает не просто согласие с мнением автора рецензии, а признание ценности самой обратной связи. Для пользователя это становится дополнительным стимулом писать более содержательные отзывы, а для исполнителя – способом выделить действительно полезные материалы без необходимости оставлять отдельный комментарий. Такая отметка не привязана к «положительным» оценкам и может быть поставлена как похвальному, так и критическому отзыву, если он аргументирован и помогает увидеть реальные сильные и слабые стороны работы. Визуальное выделение рецензий, отмеченных исполнителем, позволяет авторам качественных отзывов заявить о себе, а другим пользователям – быстрее находить мнения, на которые ориентируются сами музыканты; пример отображения такой отметки в интерфейсе системы представлен на рисунке 7.

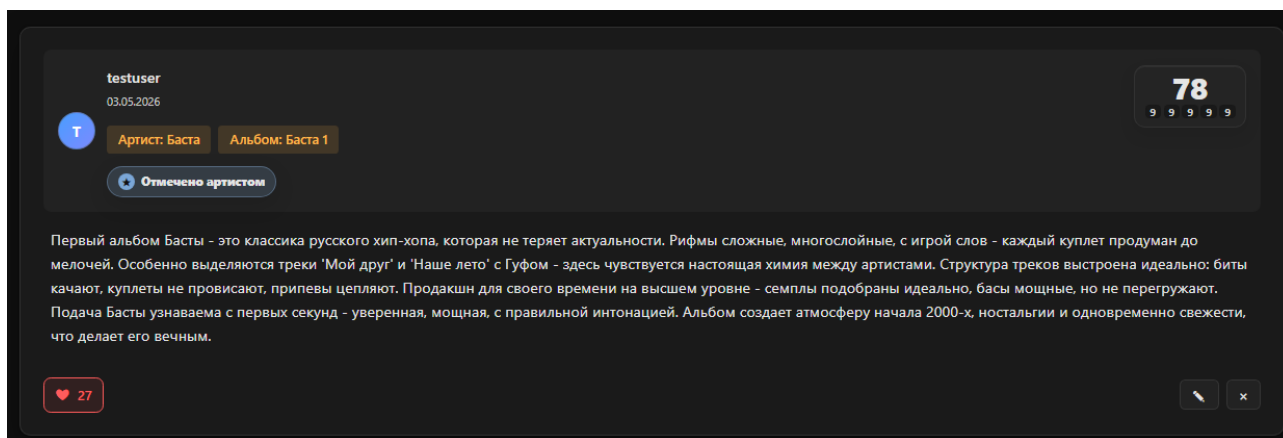


Рисунок 7 – Отображение отметки исполнителя

Ещё одним аргументом в пользу разработки специализированного web-приложения является необходимость создания удобных точек взаимодействия исполнителей со своей аудиторией в контексте рецензий. В разрабатываемой системе для музыкантов предусматриваются отдельные профили, в рамках которых консолидируется информация об исполнителе, перечень его релизов и связанные с ними пользовательские рецензии, включая отзывы, получившие наибольший отклик аудитории. Наличие такой страницы также делает систему более полезной для самих исполнителей: они получают не только список опубликованных отзывов, но и представление о том, какие релизы вызывают наибольший интерес у слушателей. За счёт этого профиль исполнителя становится не просто карточкой каталога, а отдельной точкой сбора обратной связи. Кроме того, объединение рецензий вокруг конкретного автора помогает пользователю быстрее перейти от общего впечатления о музыканте к оценке отдельных альбомов и треков. Это повышает связность пользовательского сценария и делает работу с музыкальным каталогом более осмысленной. В дальнейшем такой раздел может использоваться как основа для расширения аналитики по исполнителю: например, для отображения динамики оценок, наиболее обсуждаемых релизов и материалов, отмеченных самими музыкантами. Таким образом, профиль исполнителя дополняет обычную ленту рецензий и формирует более устойчивую связь между каталогом, отзывами и пользовательской активностью.

Такой формат позволяет слушателям воспринимать музыку не изолированно, а в связке с образом автора и реакцией сообщества, а самим исполнителям – использовать платформу как удобный канал для отслеживания обратной связи и общения с наиболее активной частью своей аудитории; пример страницы профиля исполнителя в разрабатываемой системе представлен на рисунке 8.

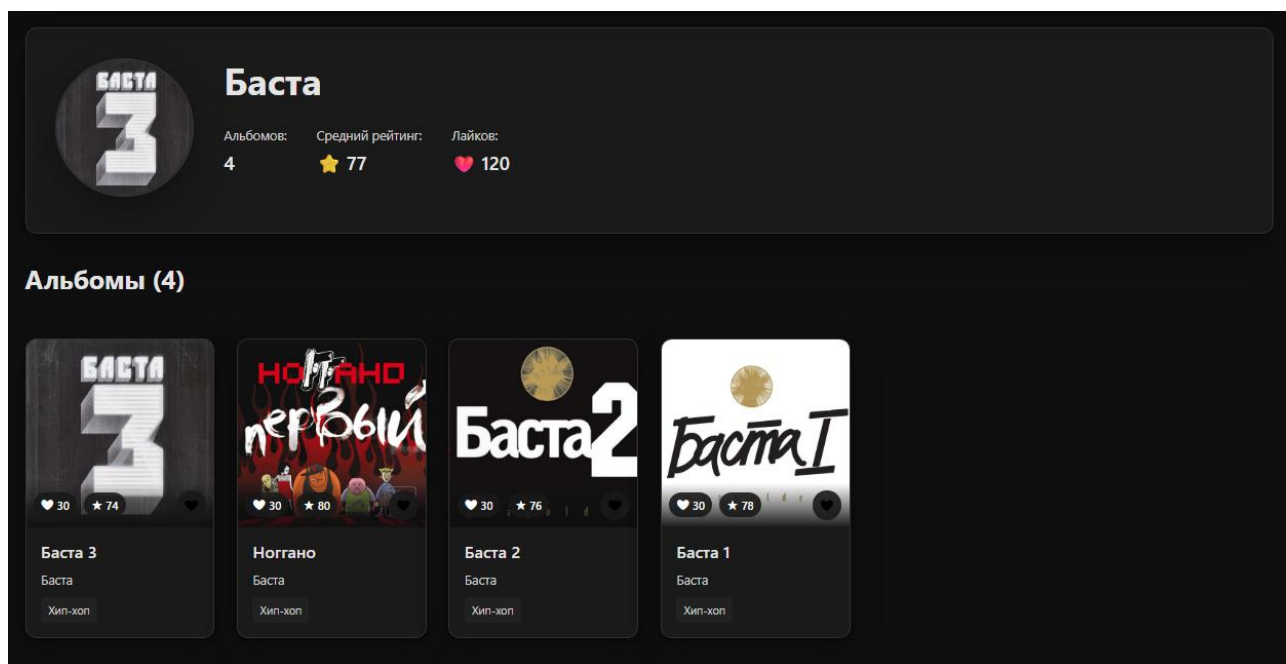


Рисунок 8 – Страница профиля исполнителя

С учётом предпочтений молодёжной аудитории важным требованием к разрабатываемой системе является лаконичный и фокусный интерфейс, в котором основное пространство экрана отведено под музыкальный материал и активность пользователей, а количество второстепенных элементов сведено к минимуму. В отличие от перегруженных рекламой и служебными блоками страниц существующих ресурсов, проектируемое web-приложение ориентируется на единый визуальный стиль, отсутствие навязчивых баннеров и удобство работы как на настольных компьютерах, так и на мобильных устройствах [7]. Пример главной страницы системы, где в центре внимания находятся актуальные релизы и раздел с лайками без избыточных отвлекающих элементов, представлен на рисунке 9.

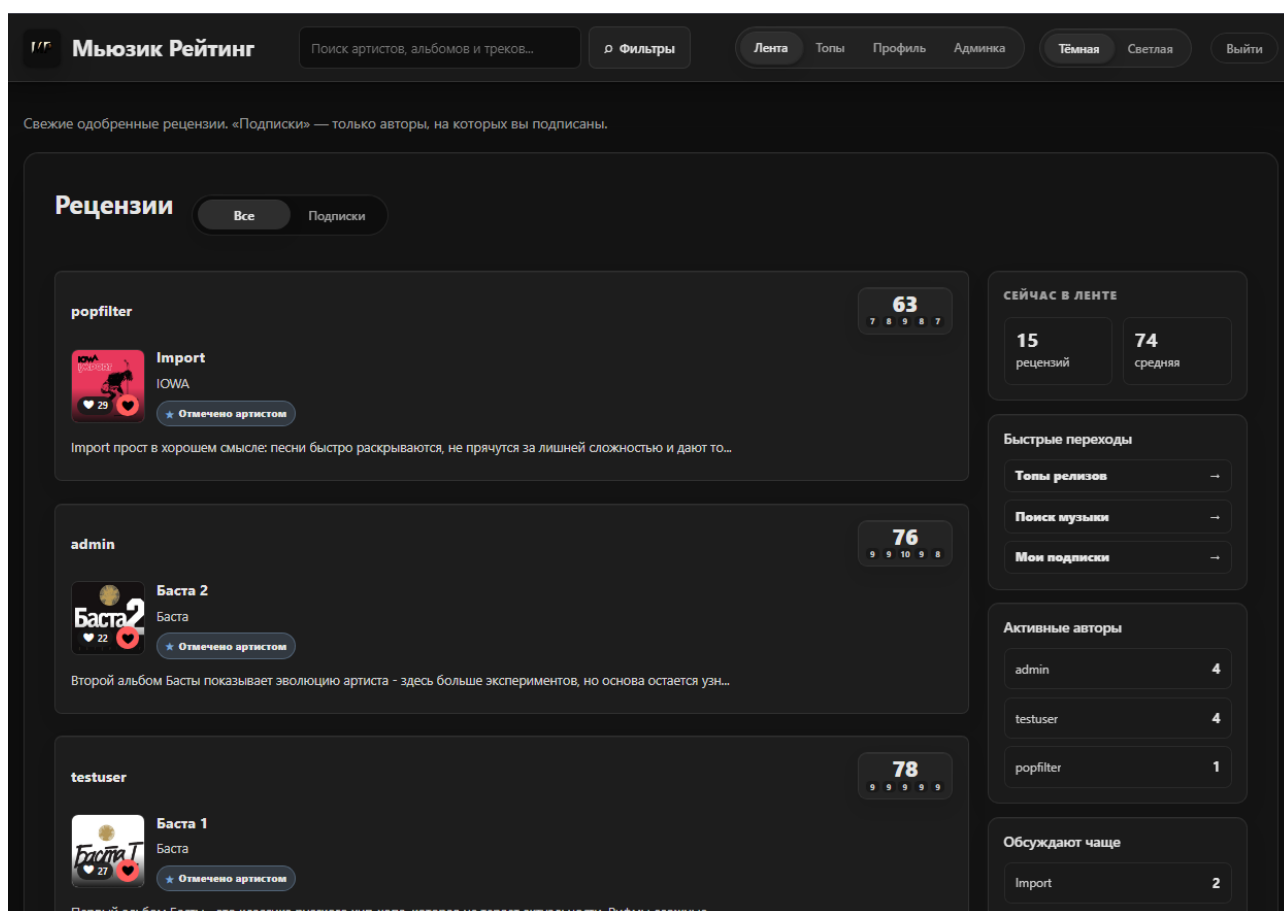


Рисунок 9 – Главная страница разрабатываемой информационной системы

Сформулированные особенности разрабатываемого web–приложения отражают запрос русскоязычной молодёжной аудитории на более осмысленное рецензирование музыки и диалог с исполнителями. Многокритериальная модель оценивания, наличие модерации текстовых отзывов, возможность отмечать содержательные рецензии со стороны автора произведения, поддержка профилей исполнителей и лаконичный интерфейс задают рамки для дальнейшего проектирования информационной системы. В следующем подразделе будут обобщены основные выводы по первой главе и уточнены положения, которые лягут в основу архитектурных и функциональных решений.

1.4 Выводы по первой главе

В результате анализа предметной области установлено, что русскоязычная музыкальная блогосфера и рынок стриминговых сервисов характеризуются высокой степенью цифровизации и значительной вовлечённостью молодёжной аудитории, однако не располагают специализированной инфраструктурой для

систематического рецензирования музыкального творчества. Существующие веб–платформы ориентированы преимущественно на потоковое прослушивание и простые количественные сигналы (прослушивания, лайки), а не на глубокий анализ произведений.

Исследование организации процесса рецензирования на действующих платформах показало преобладание одноканальных схем оценивания (звёздочные и числовые рейтинги) и слабо структурированных текстовых отзывов, оформляемых в одном свободном поле без деления на аспекты анализа. Такая модель приводит к накоплению большого числа разрозненных и неоднородных по качеству комментариев, которые затруднительно использовать в качестве надёжного ориентира для слушателей и исполнителей.

Отмечено, что модерация пользовательских рецензий на большинстве ресурсов ограничивается реакцией на явные нарушения правил общения и практически не затрагивает вопросы информативности и оригинальности. Отсутствуют удобные механизмы выделения содержательных отзывов и специальные средства работы авторов музыкальных произведений с обратной связью, что препятствует формированию устойчивого диалога между музыкантами и аудиторией.

Сравнительный анализ интерфейсов показал перегруженность многих существующих площадок служебными элементами и рекламными блоками, смещение разделов с рецензиями на периферию экрана и слабую адаптацию под мобильные сценарии использования. Для молодёжной аудитории, ориентированной на быстрый доступ к ключевым функциям и лаконичные интерфейсы, такие решения оказываются малоудобными и не стимулируют создание развёрнутых рецензий.

На основании проведённого анализа обоснована необходимость разработки специализированного веб–приложения для рецензирования музыкального творчества, ориентированного на русскоязычную молодёжную аудиторию. К ключевым особенностям проектируемой системы отнесены многокритериальная модель оценивания произведений, наличие встроенной модерации текстовых

отзывов, возможность явной отметки содержательных рецензий со стороны исполнителя, поддержка профилей музыкантов и интерфейс, фокусирующий внимание пользователя на релизах и рецензиях.

Для реализации поставленной цели, во второй главе работы требуется решить следующие задачи:

1. Сформировать функциональные и нефункциональные требования к информационной системе рецензирования музыкального творчества с учётом результатов теоретического анализа;

2. Спроектировать архитектуру web–приложения, структуру базы данных и основные пользовательские сценарии (работа слушателей, модераторов и исполнителей);

3. Разработать прототип системы, реализующий базовый набор функций, по многокритериальной оценке, модерации и взаимодействию участников, и подготовить его к последующему тестированию.

2 ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА ИНФОРМАЦИОННОЙ СИСТЕМЫ РЕЦЕНЗИРОВАНИЯ МУЗЫКАЛЬНОГО ТВОРЧЕСТВА

2.1 Архитектура веб–приложения

Разрабатываемая информационная система "Мьюзик Рейтинг" построена как веб–приложение с разделением на клиентскую часть, серверное API и уровень хранения данных. Такой подход обеспечивает независимое развитие интерфейса, бизнес–логики и базы данных, а также упрощает тестирование и развертывание системы.

Клиентская часть реализована на React и отвечает за отображение ленты рецензий, страниц альбомов и треков, профилей пользователей, форм создания рецензий, административной панели и глоссария геймификации. Взаимодействие с сервером выполняется через HTTP–запросы с использованием Axios.

Серверная часть реализована на языке Go с применением фреймворка Gin. Сервер предоставляет REST API для авторизации, работы с пользователями, альбомами, треками, жанрами, рецензиями, лайками и административными операциями. Для доступа к базе данных используется ORM–библиотека GORM.

В качестве системы управления базами данных выбрана PostgreSQL. Она обеспечивает надежное хранение связанных данных, поддержку транзакций и достаточную производительность для учебного веб–сервиса с перспективой дальнейшего развития [8].

Выбор стека обусловлен задачами проекта. React подходит для интерфейса с большим количеством интерактивных состояний: переключателями ленты, всплывающими подсказками, вкладками профиля и формами редактирования. Go и Gin выбраны для серверной части, поскольку позволяют получить компактное API–приложение с высокой скоростью работы и простой сборкой в единый исполняемый файл. PostgreSQL удобна для хранения связанных сущностей музыкального каталога, рецензий, лайков и подписок. Docker Compose и GitHub Actions дополняют стек, так как позволяют запускать систему воспроизводимо и проверять изменения до демонстрации.

Обобщенная архитектура информационной системы "Мьюзик Рейтинг" представлена на рисунке 10.

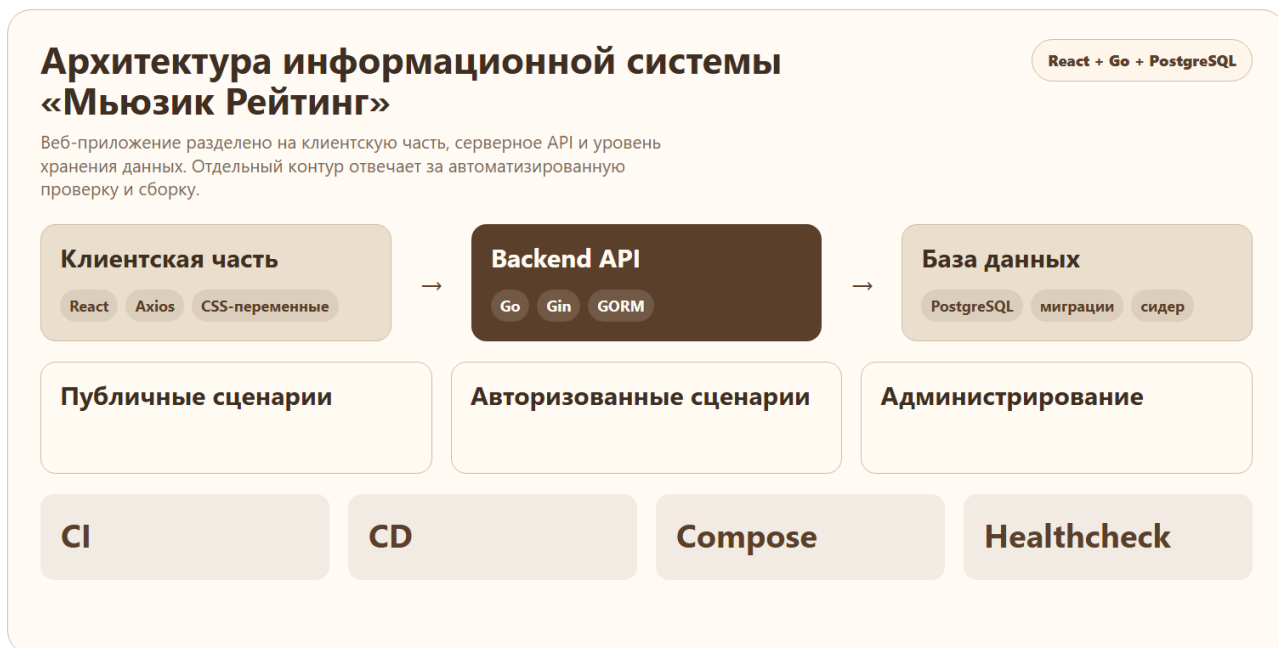


Рисунок 10 – Архитектура информационной системы «Мьюзик Рейтинг»

Представленная архитектура позволяет развивать приложение поэтапно: сначала проектировать предметную модель и API, затем подключать пользовательский интерфейс и отдельно настраивать контейнерный запуск. Это важно для выпускной работы, поскольку каждая часть системы может быть описана и проверена самостоятельно.

2.2 Функциональные требования и сценарии использования

Функциональность системы ориентирована на несколько ролей пользователей. Гость может просматривать ленту, топы, страницы альбомов и треков. Зарегистрированный пользователь дополнительно получает возможность писать рецензии, ставить оценки, лайкать материалы, подписываться на авторов и редактировать личный профиль. Администратор выполняет модерацию рецензий и управляет каталогом музыкальных релизов.

Разделение функций по ролям необходимо для соблюдения целостности пользовательского контента. Если всем посетителям предоставить одинаковый доступ к операциям изменения данных, то система быстро столкнется с некон-

тролируемым появлением некорректных рецензий, дубликатов релизов и случайных изменений каталога. Поэтому операции просмотра доступны максимально широко, а операции публикации, редактирования и модерации требуют авторизации и проверки прав [9].

Основной пользовательский сценарий строится вокруг выбора музыкального объекта и последующего анализа мнения сообщества. Пользователь может начать работу с ленты, перейти к конкретному альбому, открыть отдельный трек, изучить среднюю оценку, прочитать рецензии других участников и только после этого сформировать собственную оценку. Такой порядок соответствует естественному поведению слушателя: сначала он получает контекст, затем сравнивает разные точки зрения и только потом добавляет свой вклад.

Отдельная группа требований связана с профилем пользователя. Профиль должен отражать не только регистрационные данные, но и активность участника в сообществе: количество рецензий, полученные и поставленные лайки, любимые жанры, избранные релизы, достижения и уровень. Благодаря этому профиль выполняет не только справочную функцию, но и становится элементом мотивации, поскольку пользователь видит результат своего участия в развитии платформы.

Распределение функций между ролями пользователей приведено в таблице 1.

Таблица 1 – Функциональные требования к системе

<i>Роль пользователя</i>	<i>Основные функции</i>	<i>Ограничения доступа</i>
Гость	Просмотр ленты, топов, страниц альбомов, треков и публичных профилей	Не может создавать рецензии, ставить лайки и редактировать данные
Зарегистрированный пользователь	Создание рецензий, выставление оценок, лайки, подписки, настройка профиля и предпочтений	Не имеет доступа к модерации и управлению каталогом

Продолжение таблицы 1

<i>Роль пользователя</i>	<i>Основные функции</i>	<i>Ограничения доступа</i>
Администратор	Модерация рецензий, добавление альбомов и треков, загрузка обложек, контроль демонстрационных данных	Должен быть авторизован и иметь административную роль
Артист или отмеченный аккаунт	Публичное представление автора, авторские реакции на рецензии, участие в обратной связи	Не получает административных прав без отдельного назначения

Сценарий просмотра ленты и перехода к музыкальному объекту представлен в таблице 2.

Таблица 2 – Сценарий просмотра ленты и перехода к музыкальному объекту

<i>Действие пользователя</i>	<i>Реакция системы</i>
Пользователь открывает страницу ленты рецензий	Система загружает список одобренных рецензий, отображает автора, музыкальный объект, фрагмент текста, лайки и итоговую оценку
Пользователь переключает источник ленты на подписки	Система отображает материалы только тех авторов, на которых подписан текущий пользователь
Пользователь наводит указатель на блок оценки	Система показывает всплывающую подсказку с расшифровкой критериев оценки
Пользователь нажимает на название автора рецензии	Система выполняет переход в публичный профиль выбранного пользователя
Пользователь нажимает на карточку альбома или трека	Система открывает страницу соответствующего музыкального объекта с подробной информацией

Ключевой пользовательский сценарий начинается с выбора альбома или трека, просмотра существующих оценок и создания собственной рецензии. После публикации материал может быть отправлен на модерацию, а после одобрения появляется в общей ленте и участвует в расчете средних оценок.

Диаграмма вариантов использования, отражающая основные действия ролей, приведена на рисунке 11.



Рисунок 11 – Диаграмма вариантов использования

Диаграмма показывает, что пользовательские сценарии связаны не только с просмотром каталога, но и с созданием контента, реакциями сообщества и административным контролем. Это позволяет рассматривать систему как полноценную платформу рецензирования, а не только как справочник музыкальных релизов.

2.3 Проектирование базы данных

Модель данных включает сущности пользователей, альбомов, треков, жанров, рецензий, лайков, подписок и пользовательских предпочтений. Центральной сущностью является рецензия, которая связывает автора, оцениваемый альбом или трек, текстовое описание и набор числовых критериев.

Пользовательская модель хранит учетные данные, признак администратора, публичную информацию профиля, ссылки на социальные сети, избранные альбомы, треки и артистов. Пароль пользователя хранится не в открытом виде, а как bcrypt-хеш, что снижает риск компрометации учетных данных [10].

Альбом связан с жанром, набором треков, рецензиями и лайками. Трек связан с альбомом и может иметь несколько жанров через промежуточную таблицу.

Такая структура позволяет показывать как карточку альбома целиком, так и отдельные страницы треков с собственными оценками.

При проектировании базы данных учитывалась необходимость хранения как пользовательского контента, так и служебных связей, которые формируют поведение интерфейса. Например, таблицы лайков позволяют определить, поставил ли текущий пользователь реакцию на конкретную рецензию, а таблицы подписок используются для формирования персональной ленты. Таблица предпочтений профиля позволяет хранить выбранные пользователем любимые объекты отдельно от автоматически рассчитанных блоков.

Для демонстрационного проекта также важно обеспечить простоту восстановления состояния базы. Поэтому структура данных проектировалась так, чтобы ее можно было заполнить сидером без ручной настройки: сначала создаются жанры и музыкальные объекты, затем пользователи, далее рецензии, оценки, лайки, подписки и предпочтения. Такой порядок помогает избежать нарушения внешних ключей и делает демо–стенд воспроизводимым.

Основные сущности базы данных и их связи приведены в таблице 3.

Таблица 3 – Основные сущности базы данных

<i>Сущность</i>	<i>Назначение</i>	<i>Основные связи</i>
users	Хранение учетных данных, публичного профиля и роли пользователя.	Связана с рецензиями, которые создает пользователь.
genres	Хранение музыкальных жанров, используемых для классификации релизов и треков.	Связана с альбомами и треками.
albums	Хранение музыкальных релизов, обложек, описаний и основного жанра.	Связана с жанром, треками и рецензиями.
tracks	Хранение треков внутри альбома, их порядка, длительности и жанра.	Связана с альбомом, жанром и рецензиями.
reviews	Хранение текста рецензии, статуса модерации и критериев рейтинговой оценки.	Связана с пользователем, альбомом или треком.

Графическое представление модели данных приведено на рисунке 12.



Рисунок 12 – Логическая модель базы данных

Логическая модель отражает основные сущности предметной области и связи между ними: пользователь создает рецензии и реакции, альбом объединяет треки, а рецензия связывает автора с конкретным музыкальным объектом. Такое представление позволяет описать структуру данных на уровне предметной области и использовать ее как основу для дальнейшего физического проектирования базы данных [11].

После логической модели сформирована физическая модель базы данных. Она уточняет состав таблиц, типы данных, ключевые поля и назначение каждого атрибута. Такое описание необходимо для проверки соответствия структуры базы данных предметной области и для последующего сопоставления с миграциями PostgreSQL.

Описание таблиц физической модели приведено в таблицах 4–18.

Таблица 4 – Пользователи системы (users)

<i>Поле</i>	<i>Тип данных</i>	<i>Описание</i>
id	SERIAL	Первичный ключ пользователя.
username	VARCHAR	Уникальное имя пользователя, отображаемое в профиле и рецензиях.
email	VARCHAR	Адрес электронной почты для авторизации и идентификации пользователя.
password	VARCHAR	Хеш пароля пользователя; открытое значение пароля не хранится.
avatar_path	VARCHAR	Путь к изображению аватара пользователя.
bio	TEXT	Краткое описание профиля пользователя.
is_admin	BOOLEAN	Признак наличия административных прав.
created_at	TIMESTAMP	Дата и время регистрации пользователя.

Таблица 5 – Жанры музыкальных объектов (genres)

<i>Поле</i>	<i>Тип данных</i>	<i>Описание</i>
id	SERIAL	Первичный ключ жанра.
name	VARCHAR	Название музыкального жанра.
description	TEXT	Краткое описание жанрового направления.

Таблица 6 – Альбомы (albums)

<i>Поле</i>	<i>Тип данных</i>	<i>Описание</i>
id	SERIAL	Первичный ключ альбома.
genre_id	INTEGER	Внешний ключ на основной жанр альбома.
title	VARCHAR	Название альбома.
artist	VARCHAR	Имя исполнителя или группы.
cover_url	VARCHAR	Путь или URL обложки альбома.
description	TEXT	Краткое описание релиза.
created_at	TIMESTAMP	Дата добавления альбома в каталог.

Таблица 7 – Треки (tracks)

<i>Поле</i>	<i>Тип данных</i>	<i>Описание</i>
id	SERIAL	Первичный ключ трека.
album_id	INTEGER	Внешний ключ на альбом, в который входит трек.
title	VARCHAR	Название трека.
duration	INTEGER	Длительность трека в секундах.
track_number	INTEGER	Порядковый номер трека в альбоме.
created_at	TIMESTAMP	Дата добавления трека.

Таблица 8 – Рецензии и оценки (reviews)

<i>Поле</i>	<i>Тип данных</i>	<i>Описание</i>
id	SERIAL	Первичный ключ рецензии.
user_id	INTEGER	Внешний ключ на автора рецензии.
album_id	INTEGER	Внешний ключ на альбом; заполняется, если рецензия относится к альбому.
track_id	INTEGER	Внешний ключ на трек; заполняется, если рецензия относится к отдельному треку.
text	TEXT	Текстовая часть рецензии.

Продолжение таблицы 8

<i>Поле</i>	<i>Тип данных</i>	<i>Описание</i>
rhyme_score	INTEGER	Оценка рифм и образов.
structure_score	INTEGER	Оценка структуры и ритмики.
style_score	INTEGER	Оценка реализации стиля.
individuality_score	INTEGER	Оценка индивидуальности и харизмы.
vibe_score	INTEGER	Оценка атмосферы и общего впечатления от музыкального объекта.
total_score	INTEGER	Итоговая рейтинговая оценка, рассчитанная на основе критериев.
status	VARCHAR	Статус модерации рецензии.
created_at	TIMESTAMP	Дата создания рецензии.

Физическая модель показывает, какие данные хранятся постоянно, а какие вычисляются приложением при формировании ответа API. Например, средние оценки по отдельным критериям не сохраняются отдельными столбцами: они рассчитываются по одобренным рецензиям и возвращаются клиентской части вместе с музыкальным объектом [12].

2.4 Модель рейтинговой оценки и геймификации

2.4.1 Модель рейтинговой оценки

Для повышения информативности оценки в системе применяется не одно общее число, а набор критериев: рифмы и образы, структура и ритмика, реализация стиля, индивидуальность и харизма, атмосфера или вайб произведения. Итоговая оценка нормализуется к удобному диапазону и отображается в интерфейсе крупным числом.

В карточках рецензий рядом с итоговым баллом показываются компактные значения критериев, а при наведении появляется подсказка с объяснением состава оценки. Интерфейс не перегружает пользователя внутренними техническими коэффициентами, а описывает расчет через понятные музыкальные категории. Состав экспертной оценки приведен в таблице 14.

Таблица 14 – Критерии экспертной оценки

<i>Критерий</i>	<i>Содержание критерия</i>	<i>Назначение в интерфейсе</i>
Рифмы и образы	Оценивает выразительность текста, метафоры, запоминающиеся формулировки и работу с образностью	Показывает литературную составляющую рецензируемого материала
Структура и ритмика	Учитывает композицию трека, развитие куплетов, припевов, переходов и ритмическую устойчивость	Позволяет отделить общее впечатление от технической организации материала
Реализация стиля	Отражает, насколько произведение уверенно работает внутри выбранного жанра и эстетики	Помогает сравнивать релизы внутри жанрового контекста
Индивидуальность и харизма	Оценивает узнаваемость автора, подачу, характер исполнения и отличие от типовых решений	Подчеркивает авторскую составляющую музыкального произведения
Атмосфера и вайб	Отражает цельность эмоционального впечатления и способность релиза удерживать настроение	Используется как пользовательски понятное объяснение итогового балла

Выбранные критерии позволяют совместить содержательную и эмоциональную стороны восприятия музыки. За счёт этого оценка не сводится только к общей симпатии пользователя, а показывает, какие именно характеристики произведения повлияли на итоговый результат.

Карточка рецензии показывает итоговый балл, значения отдельных критериев и всплывающую подсказку с расшифровкой оценки. Такое представление демонстрирует, как расчетная модель встроена в повседневный пользовательский интерфейс; пример карточки приведен на рисунке 13.

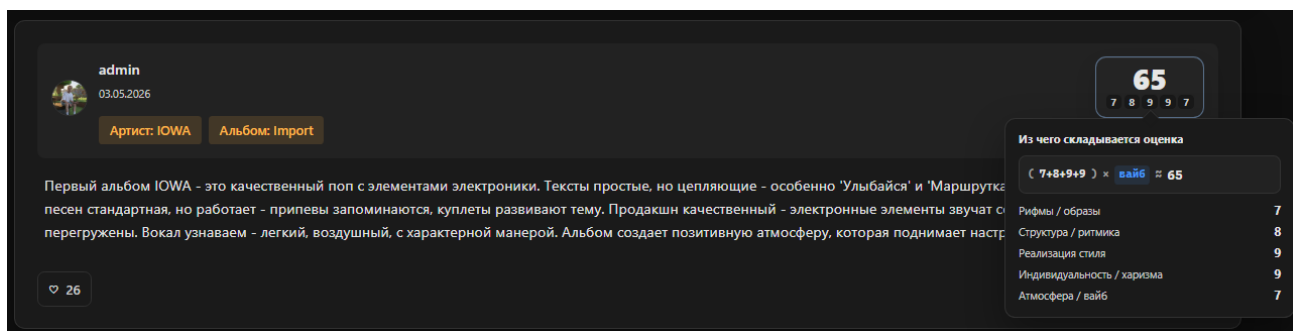


Рисунок 13 – Карточка рецензии и подсказка состава оценки

Такое отображение делает многокритериальную оценку понятной для пользователя: итоговое число остается главным визуальным ориентиром, а подробности доступны без перегрузки карточки постоянными пояснениями [13].

Для альбомов и треков в системе используется агрегированная средняя оценка, рассчитанная по опубликованным рецензиям. Она отображается на странице музыкального объекта отдельным компактным блоком, чтобы пользователь мог быстро оценить общий уровень релиза или композиции. Пример отображения средней оценки на странице музыкального объекта представлен на рисунке 14.



Рисунок 14 – Средняя оценка на странице альбома или трека

Разделение пользовательской оценки и средней оценки музыкального объекта важно для предметной области. Первая показывает мнение конкретного автора рецензии, а вторая отражает агрегированную позицию сообщества и помогает сравнивать релизы между собой.

2.4.2 Геймификация и профиль пользователя

Геймификация реализована через уровни профиля, достижения и статистику активности. Опыт начисляется за опубликованные рецензии, полученные лайки, поставленные лайки и авторские реакции. Такой механизм стимулирует пользователя писать содержательные материалы и участвовать в жизни сообщества [14].

При проектировании системы уровней важно было не связывать прогресс пользователя со строгостью или мягкостью его оценок. Средняя оценка, которую пользователь ставит релизам, отображается в статистике как характеристика вкуса, но не влияет на уровень профиля. Уровень зависит от действий, которые действительно отражают вклад в сообщество: публикация рецензий, реакция других пользователей и активность внутри ленты.

Правила начисления опыта представлены в таблице 15.

Таблица 15 – Правила начисления опыта профиля

<i>Действие пользователя</i>	<i>Начисляемый опыт</i>	<i>Обоснование</i>
Публикация одобренной рецензии	320 баллов	Рецензия является основным пользовательским вкладом в систему
Лайк, полученный на рецензию	55 баллов	Показывает, что материал оказался полезен или интересен другим участникам
Поставленный лайк	12 баллов	Поддерживает активность пользователя и участие в жизни ленты
Авторская реакция	240 баллов	Имеет повышенный вес, так как отражает внимание автора или отмеченного аккаунта

В профиле пользователя блок уровня объединяет текущий ранг, количество баллов, место в общем рейтинге и ссылку на глоссарий геймификации. Пример такого блока показан на рисунке 15.

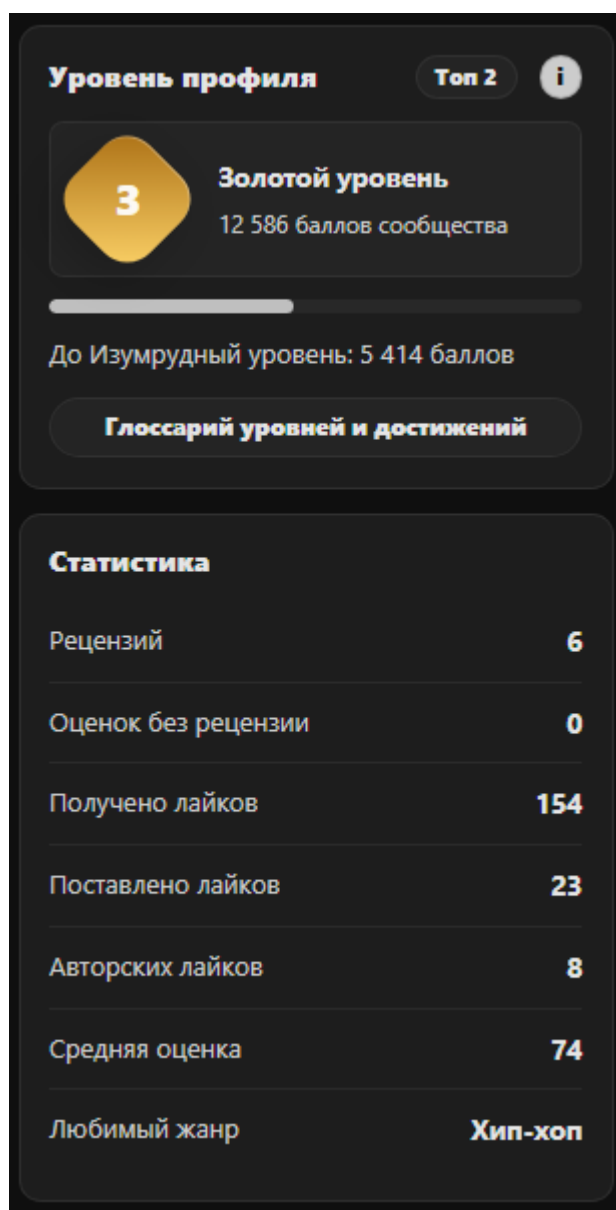


Рисунок 15 – Блок уровня профиля и статистики пользователя

Глоссарий уровней и книга достижений раскрывают правила системы в более подробном виде. Пользователь может увидеть все уровни, способы получения опыта и список достижений, включая еще не открытые. Пример такого раздела приведен на рисунке 16.

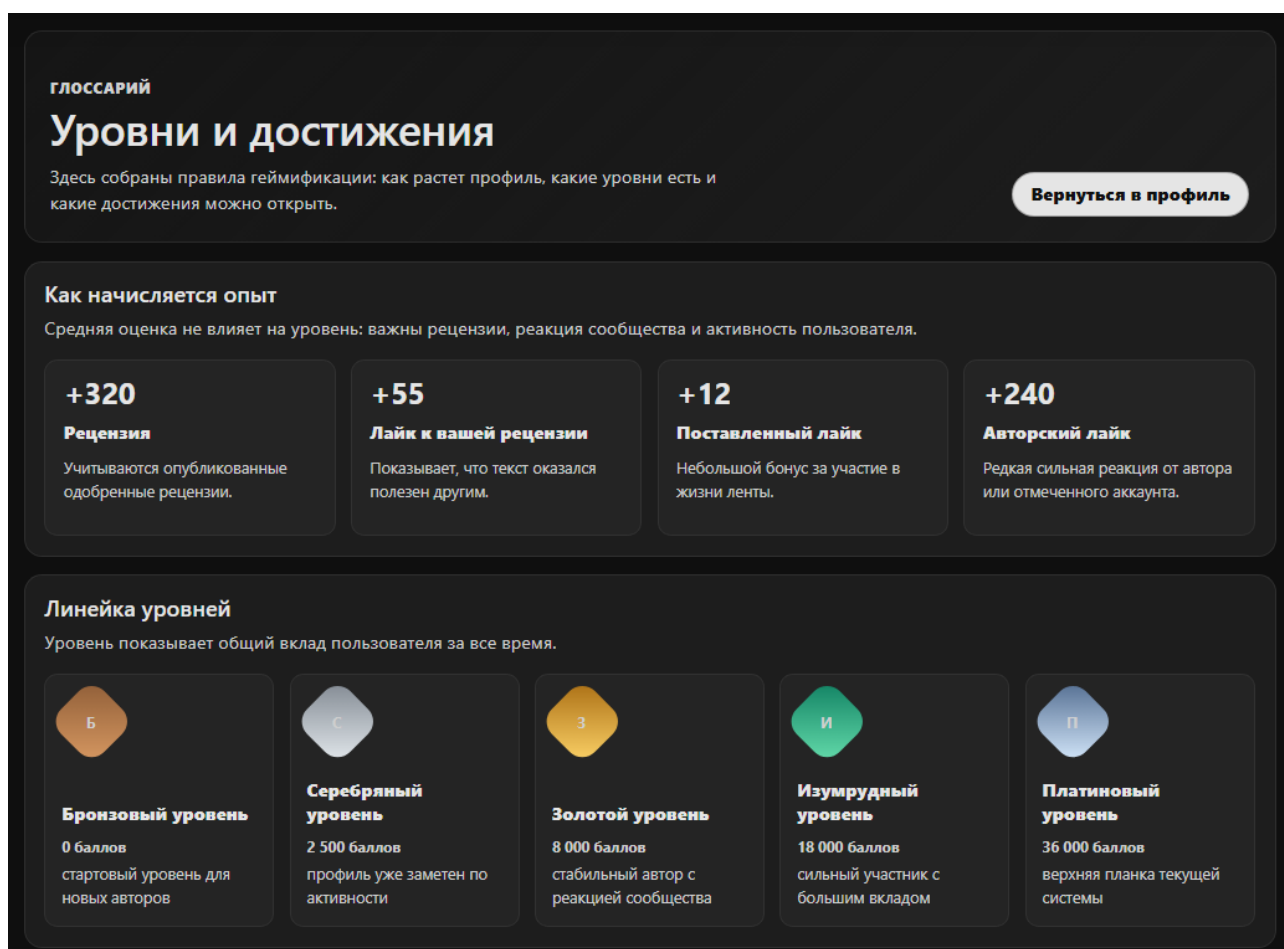


Рисунок 16 – Глоссарий уровней и книга достижений

Таким образом, геймификация выполняет не декоративную, а функциональную роль. Она помогает пользователю понимать собственный вклад, сравнивать активность с другими участниками и возвращаться к созданию рецензий.

2.5 Разработка серверной части веб-приложения

Серверная часть реализует набор контроллеров, каждый из которых отвечает за отдельную группу API-маршрутов. Контроллеры альбомов и треков обеспечивают получение каталога, просмотр детальных страниц, расчет средних оценок и работу с лайками. Контроллер рецензий отвечает за создание, изменение, удаление, одобрение и отклонение пользовательских материалов.

Для защиты административных операций применяется middleware авторизации и проверки роли администратора. Обычный пользователь может создавать рецензии и редактировать собственный профиль, но не может выполнять модерацию и управлять каталогом релизов [15].

В проект добавлен механизм загрузки пользовательских файлов: аватаров профиля и обложек альбомов. Загруженные изображения сохраняются в публичном каталоге frontend–приложения, а путь к файлу записывается в базе данных.

При реализации API учитывалось, что клиентская часть должна получать данные в форме, удобной для непосредственного отображения. Поэтому часть маршрутов возвращает не только базовые поля сущности, но и агрегированные значения: средние оценки, количество лайков, состояние реакции текущего пользователя, сведения об авторе рецензии и связанный музыкальный объект. Это уменьшает количество дополнительных запросов со стороны frontend–приложения.

Отдельное внимание уделено обработке ошибок. Для пользовательских ошибок, таких как отсутствие обязательных полей, попытка редактирования чужой записи или обращение к несуществующему объекту, сервер возвращает понятный HTTP–статус и сообщение. Такой подход облегчает последующую доработку интерфейса, поскольку клиент может показывать пользователю корректные уведомления без использования системных alert–окон.

Серверная часть также содержит операции, необходимые для администрирования каталога. При добавлении альбома сервер сохраняет основные сведения о релизе, обложку и список треков. Если длительность трека не указана вручную, она может быть заполнена автоматически в допустимом диапазоне, что удобно для демонстрационного наполнения системы и не мешает последующему редактированию данных [16].

Такая структура серверной части позволяет разделить пользовательские, административные и служебные операции, не смешивая их в одном сценарии обработки. За счёт этого API остаётся понятным для сопровождения и может расширяться без изменения уже реализованных пользовательских функций.

Основные REST API–маршруты серверной части представлены в таблице 16.

Таблица 16 – Основные REST API–маршруты

<i>Метод</i>	<i>Маршрут</i>	<i>Назначение</i>
POST	/api/register	Регистрация нового пользователя
POST	/api/login	Авторизация пользователя и получение токена доступа
GET	/api/albums	Получение списка альбомов с фильтрацией и поиском
GET	/api/albums/:id	Получение подробной информации об альбоме
POST	/api/albums/:id/reviews	Создание рецензии на альбом
GET	/api/tracks/:id	Получение подробной информации о треке
POST	/api/tracks/:id/reviews	Создание рецензии на трек
POST	/api/reviews/:id/like	Добавление или снятие лайка с рецензии
GET	/api/users/:id	Получение публичного профиля пользователя
PUT	/api/profile	Обновление данных личного профиля
PUT	/api/profile/preferences	Сохранение пользовательских предпочтений
GET	/api/admin/reviews	Получение списка рецензий для модерации
PATCH	/api/admin/reviews/:id/status	Одобрение или отклонение рецензии
POST	/api/admin/albums	Добавление альбома, обложки и списка треков

Представленные маршруты показывают внешнюю структуру взаимодействия клиента и сервера, однако не раскрывают внутреннюю последовательность обработки пользовательского действия. Для более наглядного описания логики создания рецензии была подготовлена схема, отражающая основные этапы прохождения запроса от открытия формы до сохранения результата и обновления

средней оценки музыкального объекта. Такой переход от перечня API–маршрутов к процессной схеме позволяет связать техническое описание серверной части с конкретным пользовательским сценарием.

Процесс обработки запроса создания рецензии показана на рисунке 17.

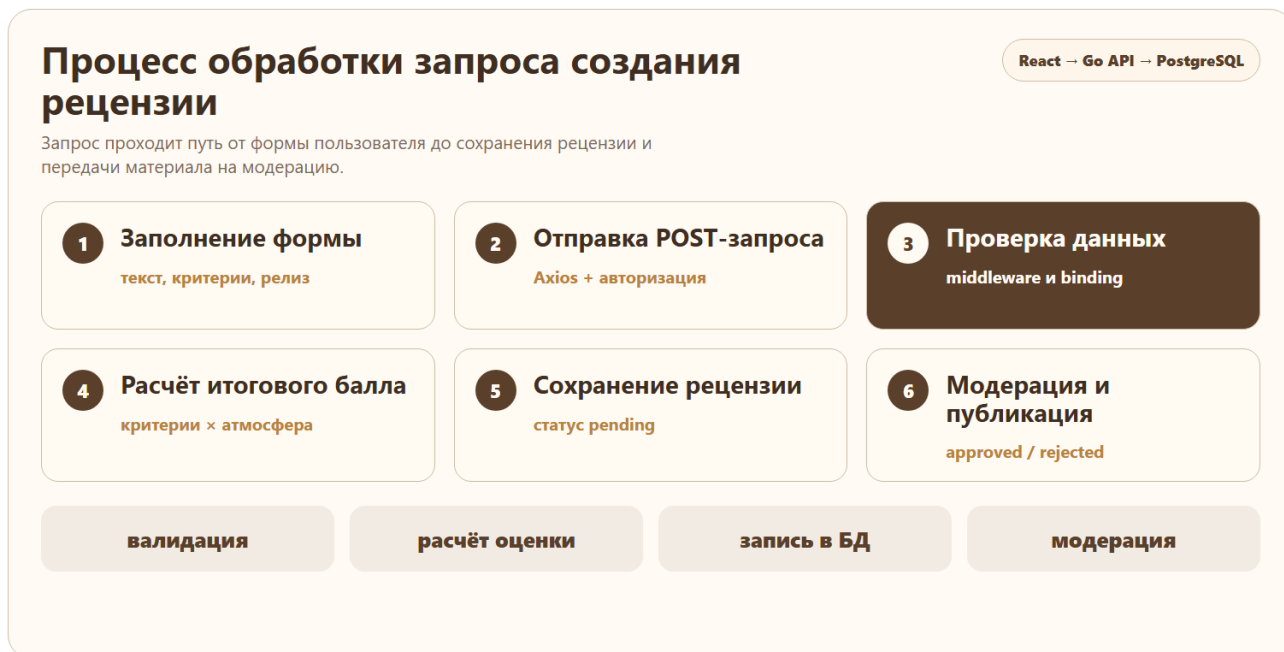


Рисунок 17 – Процесс обработки запроса создания рецензии

Данная схема показывает основные компоненты, участвующие в обработке запроса, однако не раскрывает внутренние ветвления алгоритма. Для более формального описания логики создания рецензии дополнительно построена блок–схема, отражающая проверки авторизации, корректности заполнения данных и дальнейший выбор сценария сохранения материала.

В схеме отдельно выделены этапы валидации, расчёта оценки, записи в базу данных и модерации, поскольку они определяют корректность итогового результата.

Блок–схема процесса создания рецензии представлена на рисунке 18. В ней отражены основные этапы пользовательского сценария: ввод данных, отправка формы, проверка авторизации, валидация заполненных полей, расчет итоговой оценки, сохранение данных и обновление средней оценки музыкального объекта [17].

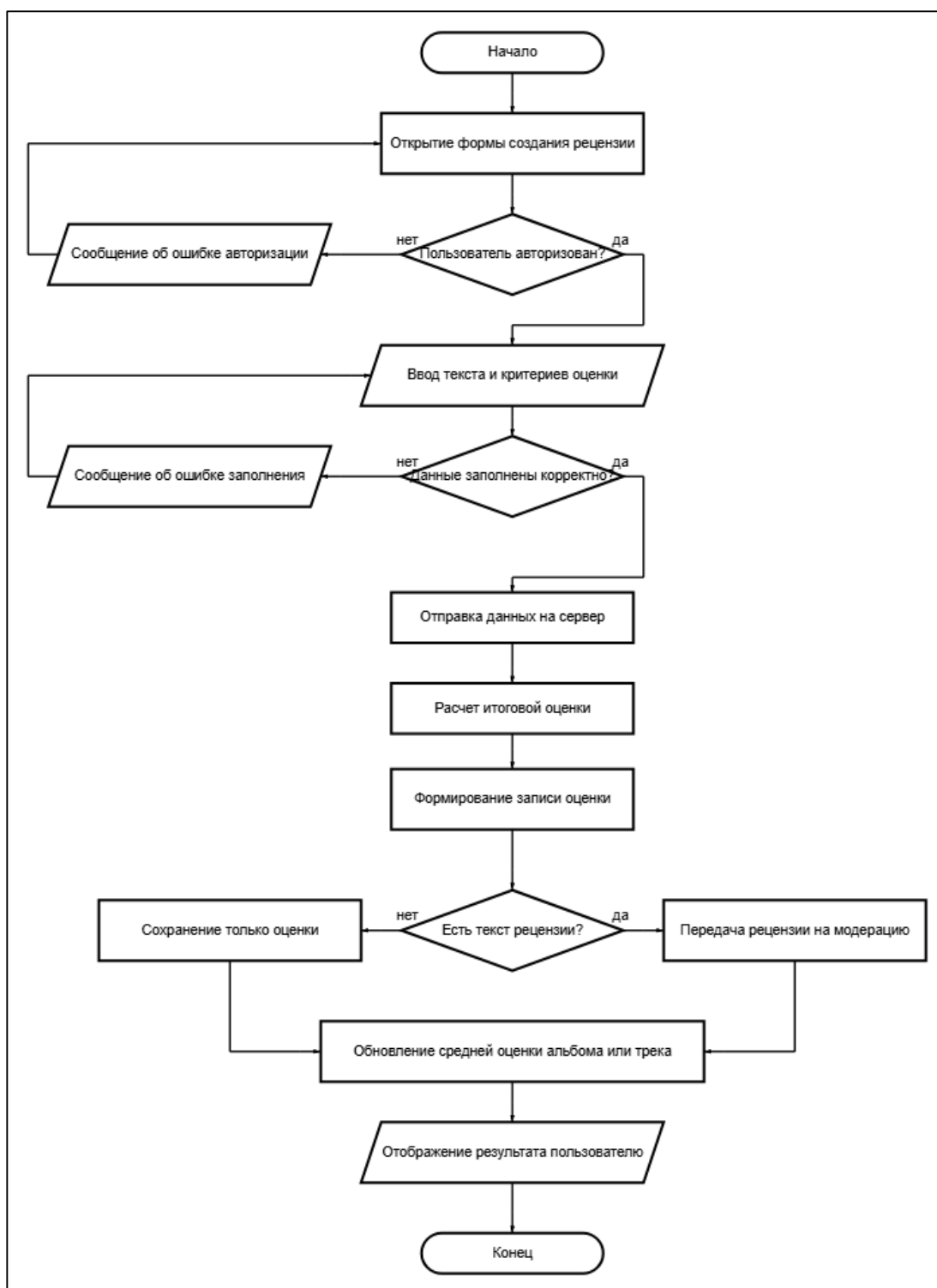


Рисунок 18 – Блок–схема процесса создания рецензии

Использование ветвлений позволяет показать, что система обрабатывает не только успешный сценарий, но и ошибочные ситуации. При отсутствии авторизации пользователь получает сообщение об ошибке и возвращается к форме, а при некорректном заполнении данных может исправить введенные значения. После успешной проверки рецензия сохраняется в базе данных, а дальнейший

сценарий зависит от наличия текстовой части: полноценная рецензия передается на модерацию, а оценка без текста используется для пересчета среднего рейтинга альбома или трека.

2.6 Разработка клиентской части веб–приложения

Клиентская часть построена на компонентном подходе React. Страницы приложения используют повторно применяемые компоненты карточек рецензий, блоков оценок, переключателей, форм, профилей и административных элементов. Это уменьшает дублирование и позволяет поддерживать единый визуальный стиль.

Особое внимание уделено профилю пользователя. В нем отображаются аватар, социальные ссылки, уровень, место пользователя в рейтинге, статистика активности, любимые артисты, альбомы и треки, а также вкладки с рецензиями, понравившимися материалами и достижениями [18].

Для интерфейса реализованы светлая и темная темы на базе CSS–переменных. Переключатели оформлены как сегментированные элементы со скользящим индикатором, что делает навигацию визуально понятной и современной.

Интерфейс проектировался как кроссплатформенный веб–интерфейс, доступный с разных типов устройств. Для этого основные страницы построены на адаптивных сетках и гибких контейнерах: на широком экране система использует пространство для параллельного отображения профиля, статистики и вкладок, а на компактном экране блоки сохраняют тот же смысл и перестраиваются в последовательный порядок. Благодаря этому приложение не привязано к одному формату экрана и может использоваться на разных устройствах без потери основных сценариев.

Компонент блока оценки был переработан отдельно, поскольку он является одним из ключевых элементов системы. Вместо набора подписанных сокращений используется крупная итоговая оценка и компактный ряд критериев. Подробная расшифровка появляется во всплывающей подсказке при наведении. Это позволяет сохранить информативность многокритериальной модели, но не перегружать карточки рецензий визуально.

Для действий пользователя используются единообразные кнопки и иконки: лайки, редактирование, удаление, поиск, переходы и социальные ссылки оформлены в рамках общей цветовой темы. Такое решение снижает ощущение случайности интерфейса и делает приложение более цельным с точки зрения пользовательского восприятия [19].

Основные маршруты клиентского приложения приведены в таблице 17.

Таблица 17 – Маршруты клиентского приложения

<i>Маршрут</i>	<i>Назначение</i>
/feed	Лента одобренных рецензий и переключение источника ленты
/tops	Раздел рейтингов и подборок музыкальных объектов
/albums/:id	Страница альбома с описанием, средней оценкой и списком треков
/tracks/:id	Страница отдельного трека с рецензиями и средней оценкой
/users/:id	Публичный профиль пользователя
/profile	Личный профиль текущего пользователя, вкладки предпочтений, рецензий, лайков и достижений
/profile/edit	Редактирование профиля, социальных ссылок и пользовательских предпочтений
/admin	Административная панель модерации и управления каталогом
/gamification	Глоссарий уровней, опыта и достижений

Карта экранов пользовательского интерфейса представлена на рисунке 19.



Рисунок 19 – Карта экранов пользовательского интерфейса

Карта экранов помогает показать навигационную целостность приложения: пользователь может перейти от ленты к музыкальному объекту, от рецензии к профилю автора, а из профиля к редактированию данных и глоссарию геймификации.

2.7 Разработка административной панели

Административная панель предназначена для обслуживания пользовательского контента и каталога музыкальных релизов. Первый раздел панели отвечает за модерацию рецензий. Администратор может переключаться между рецензиями на проверку, одобренными и отклоненными материалами, после чего принять решение по каждой записи.

Второй раздел панели предназначен для добавления нового релиза. Модератор указывает название альбома, артиста, жанр, дату релиза, описание, загружает обложку и формирует список треков. Для длительности трека предусмотрена автоматическая генерация значения в диапазоне от одной до четырех минут с возможностью ручного редактирования [20].

Наличие такого раздела переносит обслуживание каталога в пользовательский интерфейс администратора. За счет этого добавление релизов становится штатной функцией системы, а не ручной операцией с таблицами базы данных.

Форма добавления альбома и треков показана на рисунке 20.

The screenshot shows a web form titled 'КАТАЛОГ' with the subtitle 'Добавить альбом и треки'. A note states: 'Длительности можно оставить сгенерированными или поправить вручную в формате 3:24.' A 'Сохранить релиз' button is in the top right. The form is divided into two main sections. The top section is for the album, featuring a placeholder for the cover image labeled 'Обложка' with a 'Загрузить обложку' button and supported formats (JPG, PNG или WEBP до 8 МБ). To the right are input fields for 'Название альбома' (with a placeholder 'Например, Новый альбом'), 'Артист' (with a placeholder 'Имя артиста'), 'Жанр' (a dropdown menu currently showing 'Поп'), and 'Дата релиза' (a date picker showing 'ДД.ММ.ГГГГ'). Below these is a large text area for 'Описание' with a placeholder 'Коротко о релизе для страницы альбома'. The bottom section is titled 'Треклист' and includes a note '0 треков будет создано вместе с альбомом.' and a 'Добавить трек' button. It contains three rows of track input fields, each with a number (1, 2, 3), a 'Название трека' field, a duration field (2:24, 1:13, 2:42), and buttons for refresh and delete (x).

Рисунок 20 – Форма добавления альбома, обложки и треков

Данный экран демонстрирует, что административная часть охватывает не только модерацию рецензий, но и развитие музыкального каталога. Это повышает самостоятельность приложения и уменьшает зависимость от прямого вмешательства в данные.

2.8 DevOps–инструменты и воспроизводимость развертывания

Одной из важных особенностей проекта является использование DevOps–инструментов, которые повышают воспроизводимость разработки, демонстрации и развертывания веб–приложения. Для локального запуска используется Docker Compose, поднимающий три основных сервиса: PostgreSQL, backend API и frontend–приложение [21].

DevOps–подход в данном проекте рассматривается как набор практик, связывающих разработку, проверку и запуск приложения. Контейнеризация фиксирует окружение, а CI/CD–процесс автоматически проверяет, что новая версия кода собирается и может быть запущена. Для современных веб–приложений это важно, поскольку приложение обычно состоит из нескольких сервисов и зависит не только от исходного кода, но и от конфигурации окружения.

Для production–like режима подготовлен отдельный compose–файл. В нем frontend собирается как статическое приложение и обслуживается через Nginx, а backend запускается как отдельный контейнер. Для backend, frontend и базы данных заданы healthcheck–проверки, позволяющие автоматически определить готовность сервисов.

В репозитории настроен pipeline GitHub Actions. Он выполняет статическую проверку Go–кода, запуск тестов, сборку backend, установку зависимостей и сборку frontend, проверку compose–файлов, запуск production smoke test и сборку Docker–образов. При push в основную ветку образы публикуются в GitHub Container Registry.

Наличие автоматизированного pipeline снижает вероятность регрессий, упрощает демонстрацию проекта и показывает готовность приложения к сопровождению после завершения учебной разработки.

Использование контейнеризации особенно важно для выпускной квалификационной работы, поскольку демонстрация проекта часто выполняется на машине, отличающейся от рабочей среды разработчика. Docker Compose фиксирует состав сервисов, версии образов, сетевые связи и переменные окружения. Благодаря этому приложение можно развернуть одной командой без ручной установки PostgreSQL, Go, Node.js и дополнительных зависимостей [22].

Разделение конфигураций на режим разработки и production–like режим позволяет показать два разных сценария эксплуатации. В режиме разработки frontend запускается как dev–сервер с быстрым обновлением, что удобно при доработке интерфейса. В production–like режиме проверяется более реалистичная

схема: статическая сборка frontend, отдача через Nginx, отдельный backend–контейнер и база данных как независимый сервис.

CI/CD pipeline выполняет роль автоматизированного контроля качества. Если изменение нарушает сборку frontend, компиляцию backend или smoke test контейнерного окружения, ошибка фиксируется до передачи проекта дальше. Такой контур проверки показывает, что система рассматривается не только как набор исходных файлов, но и как приложение, которое можно сопровождать, проверять и повторно разворачивать.

Состав этапов CI/CD pipeline приведен в таблице 18.

Таблица 18 – Этапы CI/CD pipeline

<i>Этап pipeline</i>	<i>Выполняемые действия</i>	<i>Ожидаемый результат</i>
Backend lint and test	Статическая проверка Go–кода, запуск тестов, сборка backend–приложения	Подтверждение корректности серверной части
Frontend build	Установка зависимостей и production–сборка React–приложения	Получение статической сборки без ошибок компиляции
Compose validation	Проверка compose–файлов и переменных окружения	Подтверждение корректности контейнерной конфигурации
Production smoke test	Запуск production–like окружения и проверка готовности сервисов	Подтверждение работоспособности связки frontend, backend и PostgreSQL
Docker image build	Сборка контейнерных образов backend и frontend	Подготовка артефактов для публикации и повторного запуска

Общая последовательность CI/CD–процесса показана на рисунке 21.

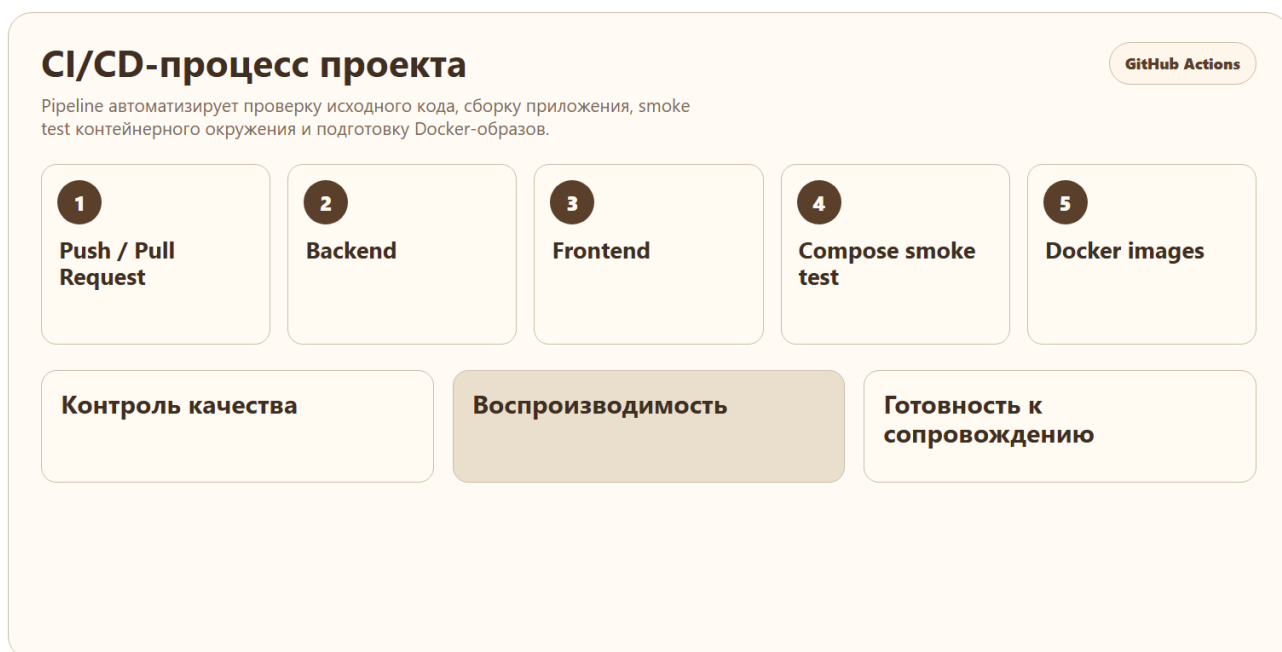


Рисунок 21 – CI/CD–процесс проекта

Использование pipeline делает проверку проекта повторяемой: один и тот же набор действий выполняется при изменениях кода и позволяет быстро выявлять ошибки сборки, тестов или контейнерной конфигурации.

2.9 Выводы по второй главе

Во второй главе была сформирована архитектура информационной системы, определены основные роли и сценарии использования, описана модель данных, разработаны механизмы рейтинговой оценки, геймификации, модерации и администрирования каталога. Также были рассмотрены инструменты контейнеризации и CI/CD, обеспечивающие воспроизводимый запуск и автоматизированную проверку проекта [23].

Архитектура системы построена с разделением на клиентскую часть, серверное API и базу данных. Такое решение соответствует распространенной практике разработки веб–приложений и позволяет независимо развивать пользовательский интерфейс, бизнес–логику и уровень хранения данных. Для выпускной квалификационной работы данное разделение также важно с методической точки зрения, поскольку позволяет последовательно рассмотреть проектирование интерфейса, серверных маршрутов, модели данных и инфраструктуры запуска.

В процессе проектирования были определены основные роли пользователей: гость, зарегистрированный пользователь, администратор и аккаунт артиста. Для каждой роли задан набор доступных функций и ограничений. Благодаря этому система не сводится к простому каталогу музыкальных объектов, а поддерживает разные сценарии взаимодействия: просмотр материалов, публикацию рецензий, выставление оценок, участие в активности сообщества и обслуживание пользовательского контента.

Разработанная модель данных отражает связи между пользователями, музыкальным каталогом, рецензиями, лайками, подписками и предпочтениями профиля. Центральной сущностью является рецензия, поскольку именно она связывает автора, музыкальный объект, текстовую оценку и числовые критерии. Такая структура обеспечивает возможность отображать оценки как на уровне альбома, так и на уровне отдельного трека, а также формировать профиль пользователя на основе его действий в системе.

Отдельное значение имеет модель рейтинговой оценки. Использование нескольких критериев позволяет сделать рецензию более содержательной, чем обычная итоговая оценка. Пользователь видит не только общий балл, но и основания, по которым он был сформирован: текстовую составляющую, ритмику, соответствие стилю, индивидуальность и атмосферу произведения. При этом интерфейс скрывает внутренние технические детали расчета и показывает результат в понятной для пользователя форме.

Рассмотренные DevOps-инструменты дополняют функциональную часть проекта. Docker Compose обеспечивает воспроизводимый запуск системы, а GitHub Actions выполняет автоматизированную проверку сборки и контейнерного окружения. В результате проект может быть представлен не только как локально работающее приложение, но и как программный продукт, для которого предусмотрены базовые механизмы проверки качества и повторного развертывания [24].

Таким образом, во второй главе были сформированы проектные решения, на основе которых реализована система "Мьюзик Рейтинг": определены роли,

модель данных, API, интерфейсные маршруты, административные функции, рейтинговая модель, геймификация и инфраструктура запуска. Эти решения создают основу для проверки работоспособности системы, которая рассматривается в третьей главе.

3 РЕЗУЛЬТАТЫ ОПЫТНОЙ ЭКСПЛУАТАЦИИ И ПРОВЕРКА РАБОТОСПОСОБНОСТИ СИСТЕМЫ

3.1 Подготовка демонстрационных данных

Для проверки работы системы подготовлены демонстрационные данные. Сидер создает администратора, обычных пользователей, дополнительные аккаунты участников сообщества, верифицированные аккаунты артистов, музыкальные альбомы, треки, рецензии, лайки и подписки.

Наличие набора демо-данных позволяет сразу после запуска показать основные сценарии без ручного наполнения базы данных.

Демонстрационные данные необходимы не только для визуального наполнения интерфейса, но и для проверки связей между сущностями. Пустое приложение не позволяет оценить корректность ленты, средних оценок, профилей, вкладок пользователя, статистики активности и административной модерации. Поэтому сидер формирует данные разных типов: музыкальный каталог, пользователей с различными ролями, рецензии с разными статусами, реакции сообщества и предпочтения профилей [25].

Для обычных пользователей создаются разные сценарии поведения. Одни пользователи активно пишут рецензии, другие чаще ставят лайки, третьи представлены как артисты или участники сообщества с меньшей активностью. Благодаря этому профиль пользователя и лента выглядят не как набор одинаковых записей, а как результат взаимодействия нескольких участников системы.

Отдельно проверяется корректность данных для артистов. Такие аккаунты могут использоваться для демонстрации авторских реакций, которые имеют больший вес в системе геймификации. Это важно для предметной области, поскольку платформа ориентирована не только на обмен мнениями между слушателями, но и на потенциальную обратную связь между авторами музыкальных произведений и аудиторией.

Данные музыкального каталога включают артистов, альбомы, треки, жанры и изображения обложек. При этом реальные названия релизов и обложки

делают демонстрацию более понятной для комиссии, поскольку интерфейс воспринимается как законченный продукт, а не как технический прототип с условными сущностями.

3.2 Пользовательский сценарий работы с системой

Пользователь может открыть ленту рецензий, перейти на страницу альбома или трека, ознакомиться со средней оценкой и подробной подсказкой по критериям, а затем создать собственную рецензию. После одобрения рецензия появляется в ленте и начинает участвовать в расчетах средних оценок.

Проверка пользовательского сценария начинается с главной ленты. На этом этапе оценивается, корректно ли отображаются опубликованные рецензии, видны ли автор, музыкальный объект, текст рецензии, лайки и итоговая оценка. Также проверяется переключение между общей лентой и лентой подписок. Такое переключение важно, поскольку оно подтверждает работу связей между пользователями и позволяет показать персонализацию контента.

Далее пользователь переходит на страницу альбома. На странице отображаются обложка, название, исполнитель, жанр, средняя оценка, описание и список треков. Для каждого трека доступен переход на отдельную страницу. Это подтверждает, что каталог работает не только как статический список, но и как связанная структура, где альбом и треки имеют собственные маршруты и могут быть самостоятельными объектами оценки [26].

После просмотра музыкального объекта пользователь создает рецензию. В форме он заполняет текст и выставляет значения по критериям. Проверка включает корректность расчета итоговой оценки, сохранение данных на сервере, отображение записи в личном профиле и изменение статистики пользователя. Если рецензия требует модерации, она не должна сразу попадать в общую ленту, что подтверждает работу жизненного цикла пользовательского контента.

Также проверяются реакции на рецензии. Пользователь может поставить лайк чужому материалу, после чего счетчик должен измениться без перезагрузки страницы. Данные о поставленных лайках отражаются во вкладке "Понрави-

лось" в профиле, а полученные лайки учитываются в статистике и системе уровней. Такой сценарий показывает связь между интерфейсными действиями и геймификацией.

Сценарий создания рецензии представлен в таблице 19.

Таблица 19 – Сценарий создания рецензии

<i>Действие пользователя</i>	<i>Реакция системы</i>
Пользователь открывает страницу альбома или трека	Система отображает информацию о музыкальном объекте, среднюю оценку и существующие рецензии
Пользователь нажимает кнопку создания рецензии	Система открывает форму с текстовым полем и критериями оценки
Пользователь заполняет текст рецензии	Система проверяет наличие обязательного текстового содержания
Пользователь выставляет оценки по критериям	Система рассчитывает итоговый балл и подготавливает данные для сохранения
Пользователь отправляет рецензию	Сервер сохраняет материал и присваивает ему статус, зависящий от правил модерации
Администратор одобряет рецензию	Рецензия появляется в общей ленте и начинает участвовать в расчете средней оценки

Сценарий редактирования профиля и предпочтений приведен в таблице 20.

Таблица 20 – Сценарий редактирования профиля и предпочтений

<i>Действие пользователя</i>	<i>Реакция системы</i>
Пользователь открывает личный профиль	Система отображает аватар, статистику, уровень, вкладки активности и любимые объекты
Пользователь переходит к редактированию профиля	Система открывает форму изменения публичных данных и социальных ссылок
Пользователь выбирает любимых артистов, альбомы и треки	Система фиксирует выбранные объекты и их порядок в блоке предпочтений

Продолжение таблицы 20

<i>Действие пользователя</i>	<i>Реакция системы</i>
Пользователь сохраняет изменения	Сервер обновляет профиль без системных alert-окон, интерфейс показывает состояние сохранения средствами приложения
Пользователь возвращается в профиль	Система отображает обновленные данные и выбранные предпочтения

Общий пользовательский сценарий работы с системой показан на рисунке 22.



Рисунок 22 – Пользовательский сценарий работы с системой

Для демонстрации пользовательской части используются несколько экранов приложения: лента рецензий, страница музыкального объекта и профиль пользователя. Первым экраном выступает лента рецензий, где видны карточки материалов, блоки оценки, лайки и переходы к музыкальным объектам; пример такого представления приведен на рисунке 23.

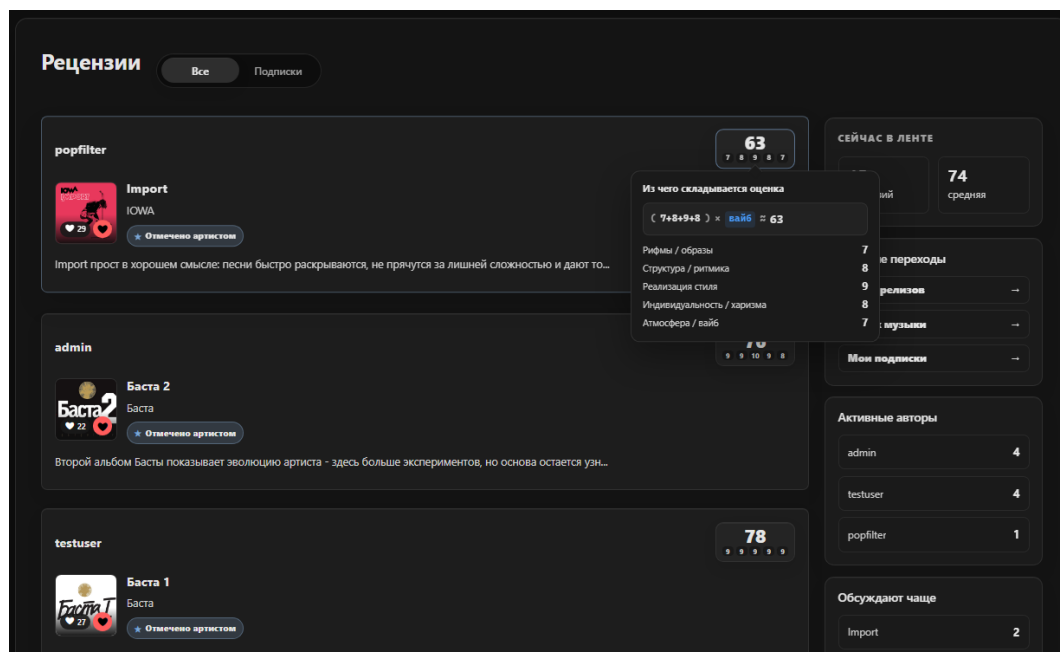


Рисунок 23 – Лента рецензий и карточка оценки

Следующий экран раскрывает работу музыкального каталога. На странице альбома отображаются сведения о релизе, средняя оценка и список треков, из которого пользователь может перейти к отдельной композиции; пример страницы приведен на рисунке 24.

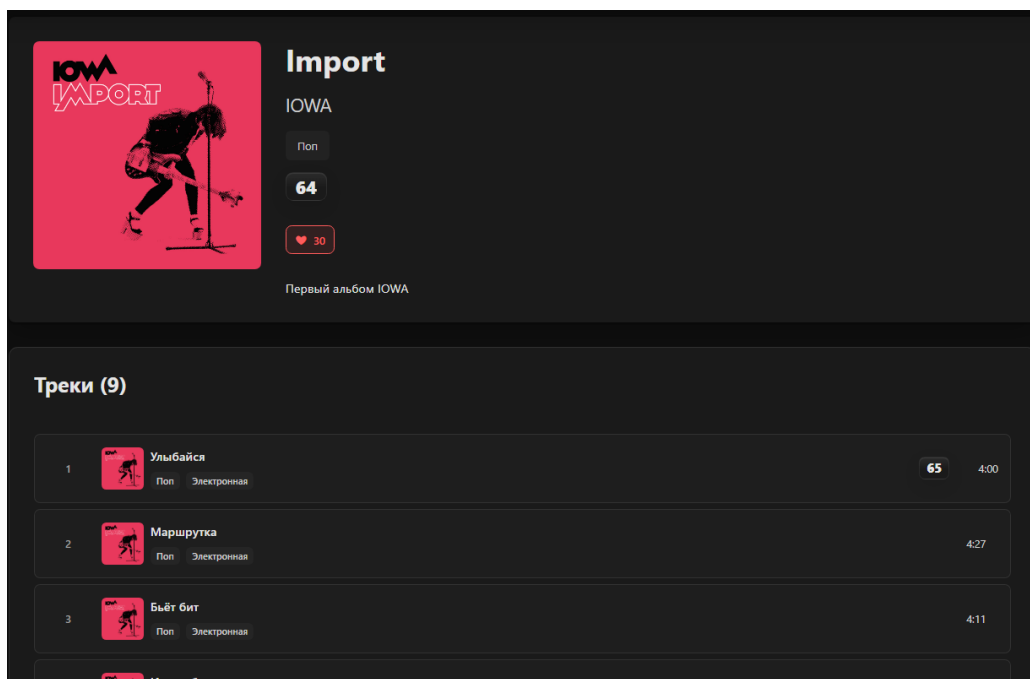


Рисунок 24 – Страница альбома со списком треков

Завершающим экраном пользовательского сценария является профиль, в котором собраны сведения об активности пользователя, его уровне, статистике,

предпочтениях и опубликованных материалах. Пример отображения профиля пользователя представлен на рисунке 25.

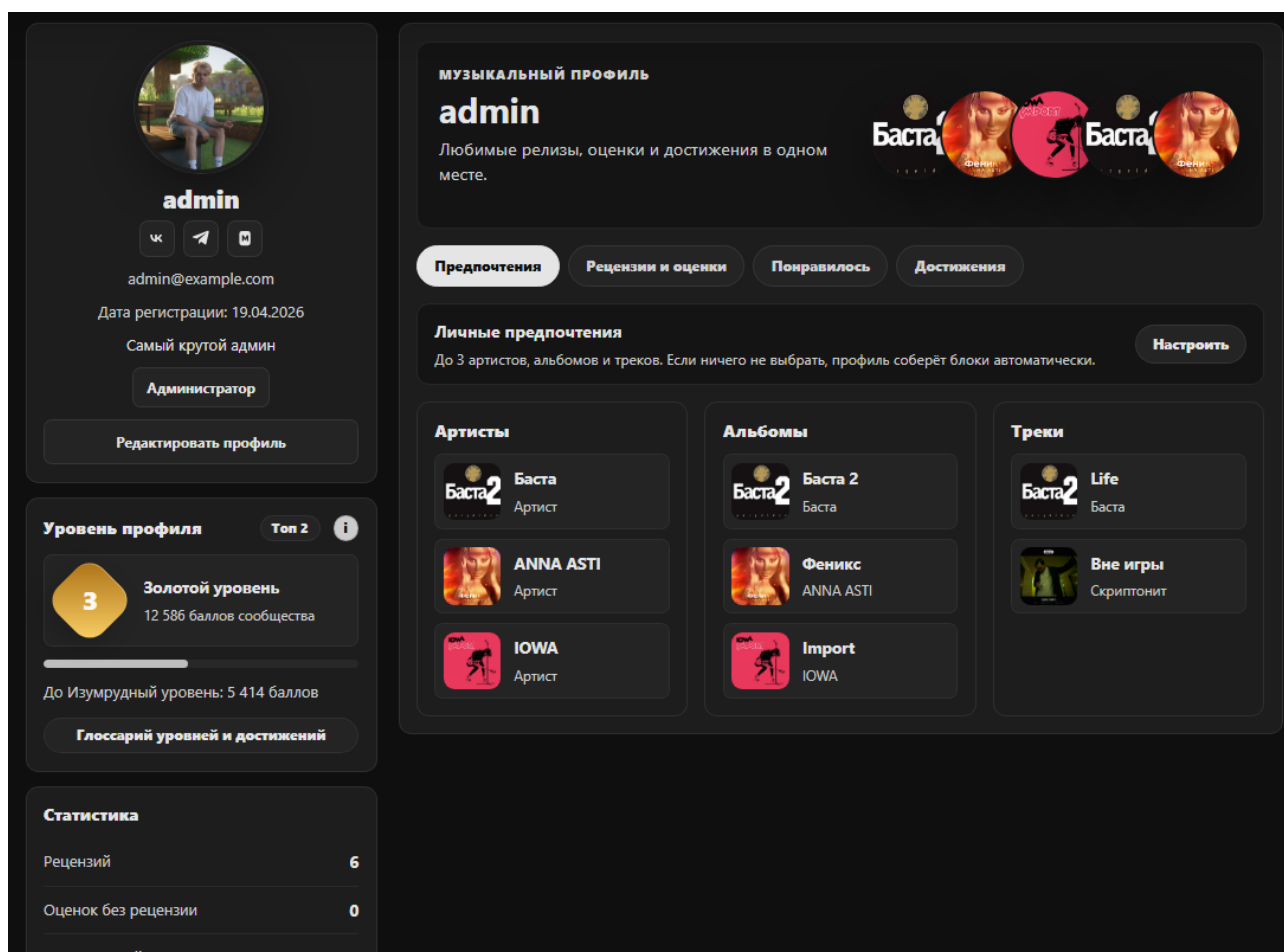


Рисунок 25 – Профиль пользователя с предпочтениями и статистикой

Профиль завершает пользовательский сценарий, поскольку показывает накопленный результат активности: рецензии, понравившиеся материалы, предпочтения, уровень и достижения.

3.3 Сценарий работы администратора

Администратор входит в панель управления, просматривает рецензии по статусам, одобряет или отклоняет материалы, а также может добавить новый альбом с треками и обложкой. Такой сценарий демонстрирует полный цикл обслуживания контента.

Проверка административного сценария начинается с авторизации под учетной записью администратора. После входа в интерфейсе появляется доступ к ад-

министративной панели. Если пользователь не обладает соответствующей ролью, переход к административным операциям должен быть запрещен серверной проверкой. Это подтверждает, что ограничение прав реализовано не только на уровне отображения кнопок, но и на уровне backend API [27].

В разделе модерации администратор просматривает рецензии, сгруппированные по статусам. Для материалов на проверке доступны действия одобрения и отклонения. После одобрения запись появляется в общей ленте и на странице связанного альбома или трека. После отклонения рецензия не участвует в публичном отображении. Таким образом проверяется полный путь пользовательского материала от создания до публикации.

Второй сценарий связан с наполнением каталога. Администратор создает новый альбом, указывает основные поля, загружает обложку и добавляет список треков. При необходимости длительность треков может быть сгенерирована автоматически, что ускоряет подготовку демонстрационных данных. После сохранения новый релиз появляется в каталоге и быть доступен для просмотра, оценки и рецензирования.

В административной панели также проверяется удобство переключения разделов. Сегментированный переключатель между страницами панели позволяет быстро перейти от модерации к добавлению релиза, не перегружая основное меню приложения. Такое решение соответствует общему подходу интерфейса, где связанные режимы работы объединяются внутри одного функционального блока.

Таким образом, административная часть системы охватывает не только контроль пользовательского контента, но и базовое сопровождение музыкального каталога. Это важно для опытной эксплуатации приложения, поскольку позволяет поддерживать актуальность данных без прямого обращения к базе данных.

Сценарий модерации рецензии представлен в таблице 21.

Таблица 21 – Сценарий модерации рецензии

<i>Действие администратора</i>	<i>Реакция системы</i>
Администратор открывает административную панель	Система проверяет роль пользователя и загружает доступные разделы панели
Администратор выбирает вкладку рецензий на проверке	Система отображает список материалов со статусом ожидания модерации
Администратор просматривает текст и оценки рецензии	Система показывает автора, музыкальный объект, критерии оценки и содержание рецензии
Администратор одобряет материал	Сервер изменяет статус рецензии, после чего она становится доступной в публичной ленте
Администратор отклоняет материал	Сервер сохраняет статус отклонения, рецензия не участвует в публичном отображении и расчетах

Сценарий добавления альбома и треков приведен в таблице 22.

Таблица 22 – Сценарий добавления альбома и треков

<i>Действие администратора</i>	<i>Реакция системы</i>
Администратор открывает раздел добавления релиза	Система отображает форму альбома и блок управления списком треков
Администратор вводит название, артиста, жанр и описание	Система проверяет обязательные поля и подготавливает объект альбома
Администратор загружает обложку	Сервер сохраняет файл и связывает путь к изображению с записью альбома
Администратор добавляет треки	Система сохраняет порядок треков и длительность, заданную вручную или сгенерированную автоматически
Администратор сохраняет релиз	Новый альбом и связанные треки становятся доступны на публичных страницах каталога

Общий сценарий работы администратора показан на рисунке 26.



Рисунок 26 – Сценарий работы администратора

Отдельно в приложении демонстрируются два административных экрана: модерация рецензий и добавление релиза в каталог. На рисунке 27 представлен экран модерации, где отображаются статусы рецензий и набор действий администратора по каждому материалу.

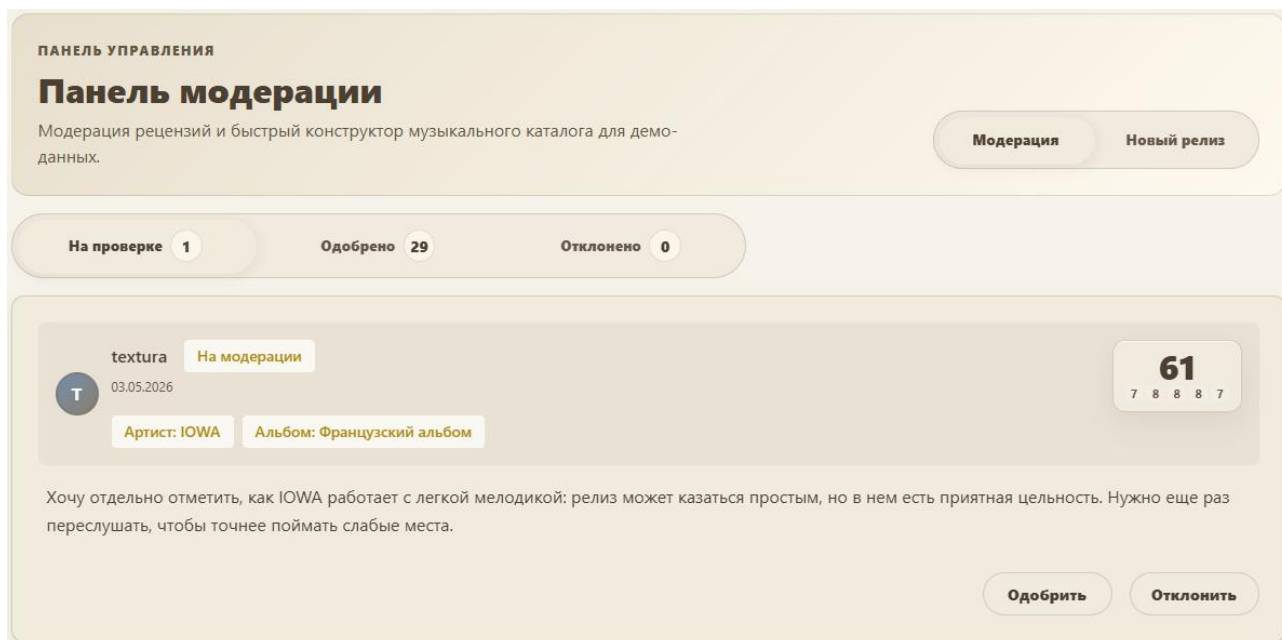


Рисунок 27 – Панель модерации рецензий

Второй административный экран в третьей главе целесообразно показать не повтором формы добавления альбома, а результатом административного действия. После сохранения релиза новый альбом должен появиться в каталоге или открываться как обычная страница музыкального объекта. Такой скриншот подтверждает, что административная операция не только заполняет форму, но и изменяет данные, доступные пользователям приложения; пример результата добавления релиза приведен на рисунке 28.

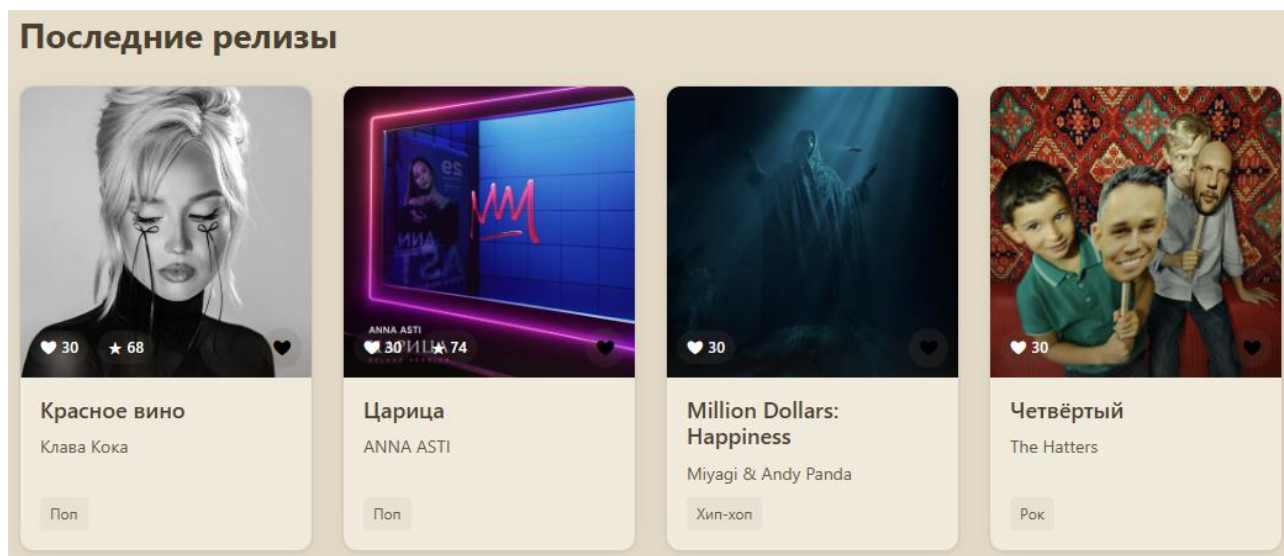


Рисунок 28 – Результат добавления альбома в каталог

Такая проверка показывает, что администратор может сопровождать систему без прямой работы с базой данных. Это важно для практической эксплуатации приложения, поскольку каталог музыкальных объектов должен пополняться через пользовательский интерфейс, а результат добавления должен быть доступен в пользовательской части системы.

3.4 Проверка качества интерфейса

Интерфейс проверяется в светлой и темной темах, на страницах ленты, профиля, альбома, трека и административной панели. Отдельно контролируются читаемость блоков оценки, корректность всплывающих подсказок и отсутствие системных alert-окон.

Проверка светлой темы выполняется отдельно, поскольку именно она чаще выявляет недостатки визуальной иерархии. На светлом фоне особенно заметны однообразные оттенки, слабые границы карточек и недостаточное различие между основным блоком и вложенными элементами. Поэтому при доработке интерфейса были усилены различия между фоном страницы, контейнерами, карточками и управляющими элементами [28].

Темная тема проверяется по другим критериям: достаточность контраста, отсутствие чрезмерно ярких акцентов, читаемость текста и аккуратность подсказок. Для всплывающих подсказок важно, чтобы они не перекрывали соседние оценки и не уходили под другие элементы интерфейса. Это особенно существенно для блока рейтинга, который часто расположен рядом с краем карточки.

Отдельно анализируется профиль пользователя. Левая колонка содержит ключевую информацию, уровень и статистику, а правая область используется для вкладок с предпочтениями, рецензиями, понравившимися материалами и достижениями. Такое построение делает профиль наполненным даже при небольшом количестве пользовательских данных и при этом не перегружает основной сценарий.

При проверке адаптивного представления основное внимание уделяется сохранению функциональности на разных устройствах. Пользователь должен иметь доступ к основным сценариям независимо от ширины экрана: просматривать ленту, переходить к альбомам и трекам, читать рецензии, работать с профилем и выполнять доступные действия. Поэтому основные страницы проверяются на корректный перенос элементов, сохранение читаемости и отсутствие наложения текста.

3.5 Проверка сборки и развертывания

Для проверки технической готовности используются команды сборки frontend, проверки backend и запуска контейнерного окружения. Данные проверки позволяют убедиться, что приложение может быть запущено не только в режиме разработки, но и в более приближенном к эксплуатации production-like окружении.

Проверка frontend включает установку зависимостей и выполнение production-сборки. Данный этап подтверждает, что клиентская часть не содержит ошибок, препятствующих сборке статических файлов. Для дипломного проекта это важно, поскольку визуально корректная работа в режиме разработки не всегда гарантирует успешную финальную сборку приложения [29].

Проверка backend включает статический анализ, запуск тестов и сборку Go-приложения. Статический анализ позволяет обнаружить часть ошибок до запуска приложения, тесты подтверждают корректность отдельных функций, а сборка показывает отсутствие критических проблем компиляции. Такой набор проверок является минимально достаточным для учебного проекта, но уже демонстрирует инженерный подход к качеству кода.

Контейнерная проверка выполняется через Docker Compose. На этом этапе поднимаются база данных, backend и frontend, после чего проверяется доступность сервисов и корректность healthcheck-состояний. Если один из контейнеров не запускается или сервис не отвечает, smoke test завершается ошибкой. Это позволяет обнаружить проблемы конфигурации окружения, которые невозможно выявить только локальной сборкой отдельных частей.

В ходе проверки также анализируется работа CI/CD pipeline. Автоматизированный сценарий выполняет последовательную проверку серверной и клиентской частей, после чего запускает контейнерное окружение и контролирует готовность сервисов. Успешное прохождение pipeline подтверждает, что исходный код, конфигурационные файлы и контейнерная сборка находятся в согласованном состоянии.

Наличие такого контура проверки повышает надежность сопровождения проекта. При внесении изменений в интерфейс, серверную часть или конфигурацию окружения ошибка может быть обнаружена до демонстрации приложения. Это особенно важно для системы, состоящей из нескольких взаимодействующих сервисов, поскольку локальная работоспособность отдельного компонента не гарантирует корректную работу всего комплекса.

Таблица 23 – Результаты проверки основных сценариев

<i>Проверяемый сценарий</i>	<i>Ожидаемый результат</i>	<i>Статус проверки</i>
Просмотр ленты рецензий	Отображаются одобренные материалы, оценки, лайки и переходы к объектам	Выполнено
Переход со страницы альбома к треку	Нажатие на строку трека открывает страницу выбранного трека	Выполнено
Создание рецензии	Рецензия сохраняется, получает статус и отображается в профиле пользователя	Выполнено
Модерация рецензии	Администратор может одобрить или отклонить пользовательский материал	Выполнено
Добавление альбома и треков	Новый релиз появляется в каталоге и доступен для рецензирования	Выполнено
Редактирование профиля	Обновляются публичные данные, социальные ссылки и предпочтения пользователя	Выполнено
Production–сборка frontend	Клиентская часть собирается без ошибок компиляции	Выполнено
Контейнерный запуск	Backend, frontend и PostgreSQL запускаются в едином compose–окружении	Выполнено

Результаты проверки показывают, что основные пользовательские, административные и инфраструктурные сценарии выполняются корректно. Наличие успешной сборки, контейнерного запуска и проверки ключевых действий под-

тверждает, что разработанное приложение можно воспроизвести в подготовленном окружении и использовать для демонстрации без ручной настройки отдельных компонентов.

3.6 Анализ результатов и ограничения

Разработанная система решает основные задачи рецензирования музыкального контента, но может быть расширена. Перспективными направлениями являются бесконечная лента, рекомендательные алгоритмы, расширенная аналитика пользовательских предпочтений, объектное хранилище для файлов и более детальная модель ролей.

По результатам опытной эксплуатации можно сделать вывод, что разработанная система обеспечивает основные процессы, заявленные в цели работы. Пользователь может просматривать музыкальный каталог, создавать рецензии, выставлять многокритериальные оценки, получать реакции на материалы и видеть собственную активность в профиле. Администратор может модерировать пользовательский контент и пополнять каталог релизов без прямого доступа к базе данных.

Сильной стороной системы является сочетание предметной логики и инженерной инфраструктуры. Многокритериальная модель оценки делает рецензии более информативными, чем простая числовая шкала. Профиль, уровни и достижения повышают вовлеченность пользователя. Контейнеризация и CI/CD pipeline повышают воспроизводимость запуска и упрощают контроль качества при изменениях.

Вместе с тем система имеет ограничения, характерные для учебного проекта. Хранение файлов в публичном каталоге frontend–приложения удобно для локальной демонстрации, но в промышленной эксплуатации предпочтительнее использовать отдельное объектное хранилище. Авторизация реализует базовую модель сессии, однако в дальнейшем ее можно расширить за счет refresh–токенов, более строгой политики хранения токенов и расширенного управления ролями.

Также перспективным направлением является развитие ленты. На текущем этапе она отображает подготовленный набор рецензий и может быть дополнена бесконечной подгрузкой при прокрутке. Такое решение улучшит восприятие большого количества материалов, но требует аккуратной реализации пагинации на сервере, сохранения состояния фильтров и контроля производительности клиентской части [30].

Дополнительное развитие может быть связано с аналитикой предпочтений. На основе оценок, жанров, лайков и избранных объектов можно формировать персональные рекомендации, подборки любимых артистов и динамику активности пользователя. Эти функции не являются обязательными для базовой версии, но логично продолжают выбранную предметную область и могут быть использованы при дальнейшем развитии проекта.

3.7 Выводы по третьей главе

В третьей главе описана опытная эксплуатация системы, рассмотрены пользовательские и административные сценарии, а также зафиксированы направления дальнейшего развития приложения.

Подготовка демонстрационных данных позволила проверить работу приложения в условиях, близких к реальному использованию. Наличие нескольких пользователей, артистов, рецензий, лайков, подписок и музыкальных релизов делает возможной проверку не только отдельных экранов, но и связей между ними. Благодаря этому были проверены лента рецензий, страницы альбомов и треков, профиль пользователя, статистика активности и административная модерация.

Пользовательский сценарий подтвердил, что система поддерживает полный путь работы с музыкальным объектом: просмотр ленты, переход к альбому или треку, изучение оценки, создание рецензии и получение реакции сообщества. Такой сценарий соответствует поставленной цели, поскольку приложение не ограничивается хранением каталога, а обеспечивает процесс содержательного рецензирования музыкального творчества.

Административный сценарий показал возможность управления качеством пользовательского контента. Администратор может просматривать материалы

по статусам, принимать решение об одобрении или отклонении рецензии, а также пополнять музыкальный каталог. Это позволяет поддерживать работоспособность системы без прямого вмешательства в базу данных и делает приложение более пригодным для дальнейшей эксплуатации.

Проверка интерфейса в светлой и темной темах показала, что основные элементы приложения сохраняют читаемость и визуальную целостность. Особое внимание было уделено блокам рейтинга, подсказкам, профилю пользователя, вкладкам и административной панели. Такие элементы важны для восприятия системы, поскольку пользовательская ценность проекта во многом определяется удобством работы с рецензиями и оценками.

Проверка сборки и контейнерного запуска подтвердила, что проект может быть воспроизведен в контейнерной среде. Это снижает зависимость от локальной конфигурации компьютера разработчика и упрощает демонстрацию проекта. Автоматизированный pipeline дополнительно подтверждает согласованность исходного кода, сборки и конфигурации запуска.

Таким образом, опытная эксплуатация показала работоспособность основных функциональных возможностей информационной системы. При этом выявлены направления дальнейшего развития: расширение механизма рекомендаций, развитие бесконечной ленты, улучшение хранения пользовательских файлов, усложнение модели ролей и добавление более подробной аналитики пользовательских предпочтений.

ЗАКЛЮЧЕНИЕ

В результате выполнения выпускной квалификационной работы была разработана информационная система рецензирования музыкального творчества, включающая клиентскую часть, серверное API, базу данных, механизмы пользовательской активности, модерации и администрирования каталога.

Практическая значимость работы заключается в создании готового учебного веб-сервиса, который демонстрирует полный цикл работы с пользовательским контентом: от публикации рецензии и расчета рейтинга до модерации и отображения результатов в пользовательском интерфейсе. Дополнительную ценность представляет настройка Docker Compose и CI/CD pipeline, обеспечивающих воспроизводимый запуск и автоматизированную проверку проекта.

В первой главе были рассмотрены теоретические основы и существующие решения в области рецензирования музыкального творчества. Анализ показал, что многие действующие платформы ориентированы преимущественно на потоковое прослушивание, краткие пользовательские отзывы или простые количественные рейтинги. При этом потребность в специализированной русскоязычной среде для структурированного анализа современной музыки остается актуальной.

Во второй главе были выполнены проектирование и разработка информационной системы. Определены роли пользователей и функциональные требования, разработана структура базы данных, описаны механизмы многокритериальной оценки, геймификации, модерации и управления музыкальным каталогом. Реализация клиентской части на React и серверной части на Go позволила разделить ответственность между интерфейсом, бизнес-логикой и хранением данных.

В третьей главе была проведена проверка работоспособности разработанной системы. Для этого подготовлены демонстрационные данные, рассмотрены пользовательские и административные сценарии, выполнена проверка интерфейса, сборки и контейнерного запуска. Полученные результаты подтверждают,

что приложение реализует основные функции, необходимые для публикации, оценки, модерации и отображения рецензий.

Особенностью разработанного проекта является наличие инфраструктурных средств, повышающих воспроизводимость и качество разработки. Использование Docker Compose позволяет развернуть приложение вместе с базой данных в едином окружении, а CI/CD pipeline обеспечивает автоматизированную проверку frontend, backend и production-like конфигурации. Для выпускной квалификационной работы это подтверждает не только реализацию пользовательских функций, но и готовность проекта к дальнейшему сопровождению.

Разработанная система имеет перспективы развития. В дальнейшем возможно добавить рекомендательные механизмы, расширенную аналитику предпочтений, бесконечную ленту рецензий, более гибкую модель ролей, интеграцию с объектным хранилищем для файлов и дополнительные инструменты взаимодействия артистов с аудиторией. Эти направления логично продолжают предметную область и могут быть реализованы на базе уже созданной архитектуры.

Таким образом, цель выпускной квалификационной работы достигнута. Разработана информационная система, которая обеспечивает структурированное рецензирование музыкального творчества, поддерживает пользовательскую активность, модерацию, элементы геймификации и воспроизводимое развертывание. Полученный результат может использоваться как основа для дальнейшего развития веб-платформы, ориентированной на осмысленное обсуждение современной музыки.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

1. Ермаков, С. Р. Веб–разработка : учебно–методическое пособие / С. Р. Ермаков. — Москва : РТУ МИРЭА, 2025. — 95 с. — ISBN 978–5–7339–2593–6. — Текст : электронный // Лань : электронно–библиотечная система. — URL: <https://e.lanbook.com/book/504852> (дата обращения: 11.05.2026). — Режим доступа: для авториз. пользователей.
2. Разработка серверной части веб–ресурса / В. В. Никулин, А. А. Олейников, А. А. Сорокин, А. В. Олейникова. — Санкт–Петербург : Лань, 2023. — 132 с. — ISBN 978–5–507–47868–2. — Текст : электронный // Лань : электронно–библиотечная система. — URL: <https://e.lanbook.com/book/356102> (дата обращения: 11.05.2026). — Режим доступа: для авториз. пользователей.
3. Баланов, А. Н. Бэкенд–разработка веб–приложений: архитектура, проектирование и управление проектами : учебное пособие для вузов / А. Н. Баланов. — 2–е изд., стер. — Санкт–Петербург : Лань, 2025. — 312 с. — ISBN 978–5–507–52472–3. — Текст : электронный // Лань : электронно–библиотечная система. — URL: <https://e.lanbook.com/book/451820> (дата обращения: 11.05.2026). — Режим доступа: для авториз. пользователей.
4. Рудалев, В. Г. Разработка веб–интерфейсов для доступа к данным : учебное пособие / В. Г. Рудалев, А. В. Дылевский. — Воронеж : ВГУ, 2017. — 35 с. — Текст : электронный // Лань : электронно–библиотечная система. — URL: <https://e.lanbook.com/book/154783> (дата обращения: 11.05.2026). — Режим доступа: для авториз. пользователей.
5. Булгакова, И. А. Разработка и прототипирование веб–сайтов и интерфейсов онлайн: технологии создания интерфейсов : учебное пособие для вузов / И. А. Булгакова. — Санкт–Петербург : Лань, 2025. — 224 с. — ISBN 978–5–507–52858–5. — Текст : электронный // Лань : электронно–библиотечная система. — URL: <https://e.lanbook.com/book/502477> (дата обращения: 11.05.2026). — Режим доступа: для авториз. пользователей.

6. Лагунова, А. Д. Архитектура интеграции : учебное пособие / А. Д. Лагунова, Д. М. Перегудова. — Москва : РТУ МИРЭА, 2024. — 100 с. — ISBN 978–5–7339–2220–1. — Текст : электронный // Лань : электронно–библиотечная система. — URL: <https://e.lanbook.com/book/421106> (дата обращения: 11.05.2026). — Режим доступа: для авториз. пользователей.

7. Леон, У. Разработка веб–приложения GraphQL с React, Node.js и Neo4j / У. Леон ; перевод с английского А. Н. Киселева. — Москва : ДМК Пресс, 2023. — 262 с. — ISBN 978–5–93700–185–6. — Текст : электронный // Лань : электронно–библиотечная система. — URL: <https://e.lanbook.com/book/314975> (дата обращения: 11.05.2026). — Режим доступа: для авториз. пользователей.

8. Калиберда, Е. А. Разработка web–приложений : учебное пособие / Е. А. Калиберда, К. В. Кравченко. — Омск : ОмГТУ, 2023. — 100 с. — ISBN 978–5–8149–3679–0. — Текст : электронный // Лань : электронно–библиотечная система. — URL: <https://e.lanbook.com/book/421766> (дата обращения: 11.05.2026). — Режим доступа: для авториз. пользователей.

9. Тузовский, А. Ф. Проектирование и разработка web–приложений : учебное пособие / А. Ф. Тузовский. — Томск : ТПУ, 2014. — 219 с. — Текст : электронный // Лань : электронно–библиотечная система. — URL: <https://e.lanbook.com/book/62933> (дата обращения: 11.05.2026). — Режим доступа: для авториз. пользователей.

10. Макарова, Т. В. Веб–дизайн : учебное пособие / Т. В. Макарова. — Омск : ОмГТУ, 2015. — 148 с. — ISBN 978–5–8149–2075–1. — Текст : электронный // Лань : электронно–библиотечная система. — URL: <https://e.lanbook.com/book/149129> (дата обращения: 11.05.2026). — Режим доступа: для авториз. пользователей.

11. Астахова, И. Ф. Проектирование баз данных : учебное пособие / И. Ф. Астахова, В. А. Чулюков, И. П. Половинкин. — Воронеж : ВГУ, 2017. — 74 с. — Текст : электронный // Лань : электронно–библиотечная система. — URL: <https://e.lanbook.com/book/154780> (дата обращения: 11.05.2026). — Режим доступа: для авториз. пользователей.

12. Горожанина, Е. И. Проектирование баз данных и баз знаний : учебное пособие / Е. И. Горожанина. — Самара : ПГУТИ, 2021. — 108 с. — Текст : электронный // Лань : электронно-библиотечная система. — URL: <https://e.lanbook.com/book/301085> (дата обращения: 11.05.2026). — Режим доступа: для авториз. пользователей.

13. Манухина, О. В. Информационные системы : учебное пособие / О. В. Манухина. — Чита : ЗабГУ, 2021. — 135 с. — ISBN 978-5-9293-2847-3. — Текст : электронный // Лань : электронно-библиотечная система. — URL: <https://e.lanbook.com/book/271508> (дата обращения: 11.05.2026). — Режим доступа: для авториз. пользователей.

14. Калгина, И. С. Интеллектуальные информационные системы : учебное пособие / И. С. Калгина. — Чита : ЗабГУ, 2023. — 123 с. — ISBN 978-5-9293-3270-8. — Текст : электронный // Лань : электронно-библиотечная система. — URL: <https://e.lanbook.com/book/438236> (дата обращения: 11.05.2026). — Режим доступа: для авториз. пользователей.

15. Медникова, О. В. Проектирование интерфейсов : учебно-методическое пособие / О. В. Медникова. — Москва : РУТ (МИИТ), 2019. — 68 с. — Текст : электронный // Лань : электронно-библиотечная система. — URL: <https://e.lanbook.com/book/175769> (дата обращения: 11.05.2026). — Режим доступа: для авториз. пользователей.

16. Диков, А. В. Web-программирование на JavaScript : учебное пособие для вузов / А. В. Диков. — Санкт-Петербург : Лань, 2026. — 436 с. — ISBN 978-5-507-54625-1. — Текст : электронный // Лань : электронно-библиотечная система. — URL: <https://e.lanbook.com/book/518225> (дата обращения: 11.05.2026). — Режим доступа: для авториз. пользователей.

17. Проектирование графических интерфейсов программных систем: практикум : учебное пособие / составители Р. А. Ещенко [и др.]. — Хабаровск : ДВГУПС, 2024. — 117 с. — Текст : электронный // Лань : электронно-библиотечная система. — URL: <https://e.lanbook.com/book/506859> (дата обращения: 01.05.2026). — Режим доступа: для авториз. пользователей.

18. Сейерс, Э. Х. Docker на практике / Э. Х. Сейерс, А. Милл ; перевод с английского Д. А. Беликов. — Москва : ДМК Пресс, 2020. — 516 с. — ISBN 978–5–97060–772–5. — Текст : электронный // Лань : электронно–библиотечная система. — URL: <https://e.lanbook.com/book/131719> (дата обращения: 11.05.2026). — Режим доступа: для авториз. пользователей.

19. Игнатьев, А. В. Тестирование программного обеспечения : учебное пособие для вузов / А. В. Игнатьев. — 4–е изд., стер. — Санкт–Петербург : Лань, 2025. — 56 с. — ISBN 978–5–507–50858–7. — Текст : электронный // Лань : электронно–библиотечная система. — URL: <https://e.lanbook.com/book/481331> (дата обращения: 11.05.2026). — Режим доступа: для авториз. пользователей.

20. Бубнов, А. А. Тестирование программного обеспечения : учебное пособие / А. А. Бубнов, С. А. Бубнов, В. В. Тишкина. — Рязань : РГРТУ, 2024. — 164 с. — ISBN 978–5–7722–0421–4. — Текст : электронный // Лань : электронно–библиотечная система. — URL: <https://e.lanbook.com/book/494540> (дата обращения: 11.05.2026). — Режим доступа: для авториз. пользователей.

21. Баланов, А. Н. DevOps: интеграция и автоматизация : учебное пособие для вузов / А. Н. Баланов. — 3–е изд., стер. — Санкт–Петербург : Лань, 2026. — 240 с. — ISBN 978–5–507–54640–4. — Текст : электронный // Лань : электронно–библиотечная система. — URL: <https://e.lanbook.com/book/509963> (дата обращения: 11.05.2026). — Режим доступа: для авториз. пользователей.

22. Турнецкая, Е. Л. Программная инженерия. Тестирование и контроль качества программного обеспечения : учебное пособие для вузов / Е. Л. Турнецкая, А. В. Аграновский. — Санкт–Петербург : Лань, 2025. — 172 с. — ISBN 978–5–507–51677–3. — Текст : электронный // Лань : электронно–библиотечная система. — URL: <https://e.lanbook.com/book/455672> (дата обращения: 11.05.2026). — Режим доступа: для авториз. пользователей.

23. Баланов, А. Н. Комплексное руководство по разработке: от мобильных приложений до веб–технологий : учебное пособие для вузов / А. Н. Баланов. — 2–е изд., стер. — Санкт–Петербург : Лань, 2025. — 412 с. — ISBN 978–5–507–53193–6. — Текст : электронный // Лань : электронно–библиотечная система. —

URL: <https://e.lanbook.com/book/478178> (дата обращения: 11.05.2026). — Режим доступа: для авториз. пользователей.

24. Позевалкин, В. В. Разработка веб-приложений на основе клиентских каркасов и библиотек : учебное пособие / В. В. Позевалкин, Н. Ф. Панова. — Оренбург : ОГУ, 2025. — 191 с. — ISBN 978-5-7410-3380-7. — Текст : электронный // Лань : электронно-библиотечная система. — URL: <https://e.lanbook.com/book/502621> (дата обращения: 11.05.2026). — Режим доступа: для авториз. пользователей.

25. Баланов, А. Н. Прототипирование и разработка пользовательского интерфейса: оптимизация UX : учебное пособие для вузов / А. Н. Баланов. — 2-е изд., стер. — Санкт-Петербург : Лань, 2026. — 220 с. — ISBN 978-5-507-55007-4. — Текст : электронный // Лань : электронно-библиотечная система. — URL: <https://e.lanbook.com/book/515090> (дата обращения: 11.05.2026). — Режим доступа: для авториз. пользователей.

26. Борисенков, Д. В. Расширенные возможности языка SQL : учебное пособие / Д. В. Борисенков, Ю. А. Крыжановская. — Воронеж : ВГУ, 2021. — 31 с. — Текст : электронный // Лань : электронно-библиотечная система. — URL: <https://e.lanbook.com/book/454688> (дата обращения: 11.05.2026). — Режим доступа: для авториз. пользователей.

27. Гринченко, Н. Н. Базы данных. Программирование на SQL : учебник / Н. Н. Гринченко, Н. И. Хизриева. — Рязань : РГРТУ, 2023. — 240 с. — ISBN 978-5-907535-77-0. — Текст : электронный // Лань : электронно-библиотечная система. — URL: <https://e.lanbook.com/book/439604> (дата обращения: 11.05.2026). — Режим доступа: для авториз. пользователей.

28. Зубкова, Т. М. Технология разработки программного обеспечения : учебное пособие / Т. М. Зубкова. — Санкт-Петербург : Лань, 2022. — 324 с. — ISBN 978-5-8114-3842-6. — Текст : электронный // Лань : электронно-библиотечная система. — URL: <https://e.lanbook.com/book/206882> (дата обращения: 11.05.2026). — Режим доступа: для авториз. пользователей.

29. Куликова, И. В. Программное обеспечение облачных и распределённых вычислительных систем : учебное пособие / И. В. Куликова. — Москва : РТУ МИРЭА, 2025 — Часть 1 — 2025. — 86 с. — ISBN 978-5-7339-2497-7. — Текст : электронный // Лань : электронно-библиотечная система. — URL: <https://e.lanbook.com/book/493544> (дата обращения: 11.05.2026). — Режим доступа: для авториз. пользователей.

30. Ефимов, С. В. Программное обеспечение автоматизированных систем управления технологическими процессами : учебное пособие / С. В. Ефимов. — Томск : ТПУ, 2020. — 128 с. — ISBN 978-5-4387-0919-0. — Текст : электронный // Лань : электронно-библиотечная система. — URL: <https://e.lanbook.com/book/246113> (дата обращения: 11.05.2026). — Режим доступа: для авториз. пользователей.

ПРИЛОЖЕНИЕ А

«ЛИСТИНГ КОНФИГУРАЦИИ CI/CD ПАЙПЛАЙНА ПРОЕКТА»

```
1. name: CI
2.
3. on:
4.   push:
5.     branches: [ main, master ]
6.   pull_request:
7.     branches: [ main, master ]
8.
9. env:
10.   FORCE_JAVASCRIPT_ACTIONS_TO_NODE24: true
11.
12. jobs:
13.   backend:
14.     name: Backend lint & test & build
15.     runs-on: ubuntu-latest
16.     defaults:
17.       run:
18.         working-directory: backend
19.
20.     steps:
21.       - name: Checkout
22.         uses: actions/checkout@v4
23.
24.       - name: Set up Go
25.         uses: actions/setup-go@v5
26.         with:
27.           go-version: '1.22'
28.           cache-dependency-path: backend/go.sum
29.
30.       - name: Download dependencies
31.         run: go mod download
32.
33.       - name: Go vet
34.         run: go vet ./...
35.
36.       - name: Go test
37.         run: go test ./...
38.
39.       - name: Build backend binary
40.         run: go build ./...
41.
```

```
42.   frontend:
43.     name: Frontend build
44.     runs-on: ubuntu-latest
45.     defaults:
46.       run:
47.         working-directory: frontend
48.
49.     steps:
50.       - name: Checkout
51.         uses: actions/checkout@v4
52.
53.       - name: Set up Node
54.         uses: actions/setup-node@v4
55.         with:
56.           node-version: '18'
57.           cache: 'npm'
58.           cache-dependency-path: 'frontend/package-lock.json'
59.
60.       - name: Install dependencies
61.         run: npm install --no-fund --no-audit
62.
63.       - name: Build
64.         env:
65.           REACT_APP_API_URL: http://localhost:8080/api
66.         run: npm run build
```

ПРИЛОЖЕНИЕ Б

«ЛИСТИНГ КОДА АЛГОРИТМА РАСЧЕТА РЕЙТИНГОВОЙ ОЦЕНКИ»

```
1. export const convertAtmosphereToMultiplier = (atmosphereRating) => {
2.   const step = 0.6072 / 9;
3.   return 1.0000 + (atmosphereRating - 1) * step;
4. };
5.
6. export const calculateFinalScore = (
7.   ratingRhymes,
8.   ratingStructure,
9.   ratingImplementation,
10.  ratingIndividuality,
11.  atmosphereRating
12. ) => {
13.   const baseScore =
14.     ratingRhymes +
15.     ratingStructure +
16.     ratingImplementation +
17.     ratingIndividuality;
18.
19.   const atmosphereMultiplier =
20.     convertAtmosphereToMultiplier(atmosphereRating);
21.
22.   const score = baseScore * 1.4 * atmosphereMultiplier;
23.   return Math.round(score);
24. };
25.
26. export const validateRating = (rating) => {
27.   return rating >= 1 && rating <= 10;
28. };
29.
30. export const formatScore = (score) => {
31.   return Math.round(score).toString();
32. };
```

ПРИЛОЖЕНИЕ В

«ЛИСТИНГ КОДА РАСЧЕТА ОПЫТА И СТАТИСТИКИ ПРОФИЛЯ ПОЛЬЗОВАТЕЛЯ»

```
1. type UserStats struct {
2.     TotalReviews      int      `json:"total_reviews"`
3.     RatingsWithoutReview int64   `json:"ratings_without_review"`
4.     AvgScore           float64 `json:"avg_score"`
5.     TotalLikesReceived int64   `json:"total_likes_received"`
6.     TotalLikesGiven    int64   `json:"total_likes_given"`
7.     AuthorLikesReceived int64   `json:"author_likes_received"`
8.     TopGenre           string  `json:"top_genre"`
9. }
10.
11. func calculateProfilePoints(stats UserStats) int {
12.     return int(math.Round(
13.         float64(stats.TotalReviews)*320 +
14.         float64(stats.TotalLikesReceived)*55 +
15.         float64(stats.TotalLikesGiven)*12 +
16.         float64(stats.AuthorLikesReceived)*240,
17.     ))
18. }
19.
20. func (uc *UserController) CalculateProfileRank(
21.     userID uint,
22.     userStats UserStats,
23. ) int {
24.     userPoints := calculateProfilePoints(userStats)
25.
26.     var users []models.User
27.     if err := uc.DB.Select("id").Find(&users).Error; err != nil {
28.         return 0
29.     }
30.
31.     rank := 1
32.     for _, candidate := range users {
33.         if candidate.ID == userID {
34.             continue
35.         }
36.
37.         candidateStats := uc.CalculateUserStats(candidate.ID)
38.         if calculateProfilePoints(candidateStats) > userPoints {
39.             rank++

```

```

40.         }
41.     }
42.
43.     return rank
44. }
45.
46. func (uc *UserController) CalculateUserStats(userID uint) UserStats
47. {
48.     var stats UserStats
49.     var reviews []models.Review
50.     uc.DB.
51.         Where("user_id = ? AND status = ?", userID, models.Re-
viewStatusApproved).
52.         Find(&reviews)
53.
54.     stats.TotalReviews = len(reviews)
55.
56.     if stats.TotalReviews > 0 {
57.         var totalScore float64
58.         for _, review := range reviews {
59.             totalScore += review.FinalScore
60.         }
61.
62.         stats.AvgScore =
63.             math.Round(totalScore/float64(stats.TotalReviews)*10) /
10
64.     }
65.
66.     var reviewIDs []uint
67.     for _, review := range reviews {
68.         reviewIDs = append(reviewIDs, review.ID)
69.     }
70.
71.     if len(reviewIDs) > 0 {
72.         uc.DB.
73.             Model(&models.ReviewLike{}).
74.             Where("review_id IN ?", reviewIDs).
75.             Count(&stats.TotalLikesReceived)
76.
77.         uc.DB.
78.             Model(&models.ReviewLike{}).
79.             Joins("JOIN users ON users.id = review_likes.user_id").
80.             Where(

```

```

81.             "review_likes.review_id IN ? AND users.is_veri-
            fied_artist = ?",
82.             reviewIDs,
83.             true,
84.         ).
85.         Count(&stats.AuthorLikesReceived)
86.     }
87.
88.     uc.DB.
89.         Model(&models.ReviewLike{}).
90.         Where("user_id = ?", userID).
91.         Count(&stats.TotalLikesGiven)
92.
93.     genreStats := uc.CalculateGenreStats(userID)
94.     if len(genreStats) > 0 {
95.         stats.TopGenre = genreStats[0].Name
96.     }
97.
98.     return stats
99. }

```